

Manual testing guide

QuickXchange / Rook exchange platform • Code-inspected edition • 27 September 2026

This document is a human execution guide derived from the present React web artifacts, Express routes, OpenAPI contract, and browser/API test fixtures. It specifies actions and observable results; it is not a record that these tests passed. Dynamic labels and available markets depend on current Admin configuration. Select a real enabled route shown in the test environment rather than assuming a particular currency is live.

Before you begin

- Test only in an isolated development or staging environment. Never send real money, rotate live provider credentials, delete real accounts, run production reconciliation, or publish configuration as part of this guide. Use sandbox provider responses, fixtures, and an approved email sink for delivery scenarios.
- Keep separate identities: signed-out guest; customer A; customer B (foreign-order checks); referred customer; Admin staff with view-only permissions; Admin operator with explicitly assigned manage permissions; Owner with enrolled second factor; unlinked Telegram chat; linked Telegram chat. Use a private chat for bot tests.
- Seed or identify one valid Manual Swap route, one valid Quickex Convert route, fiat and crypto settlement routes, a memo-required network, an active and terminal order of each type, an archived manual order, a published and draft article, one affiliate request, and one safe synthetic monitoring match. Use current Admin catalog values, not production addresses.
- For every scenario, record actual result, pass/fail, environment, account/role, selected route, timestamp, order ID if safe to disclose internally, and screenshot or redacted request/response. Never copy bearer tokens, customer addresses, webhook signatures, or credentials into a shared defect report.
- Check desktop (~1440px), tablet (~768px), mobile (~390px), and a compact phone (~320px) where a scenario calls for responsive behavior. Repeat keyboard Tab/Enter/Escape checks for menus, drawers, accordions, and selectors.
- Screen routes in this guide are relative to the web widget preview base path; API paths are relative to the API base (normally /api in the routed preview). Telegram Mini App routes are relative to its own artifact base and require Telegram initData for protected views. Preview/staging URLs must be obtained from the current environment, never inferred from a production domain.
- For provider outages, validation errors, permission denials and time-sensitive states, use a controlled test fixture or staging simulator. Do not weaken live auth, forge real deposits, or intentionally disrupt production dependencies to make a negative case happen. If a prerequisite cannot safely be met, mark the case BLOCKED with the reason rather than PASS.

Roles and result rules

- Guest: landing/information, markets, public articles, quote, checkout and capability-based order tracking. A tracking token is not account ownership.
- Customer: Clerk sign-in/sign-up, own dashboard/orders/preferences, optional affiliate area, and website-mediated Telegram linking. Customer B must not see customer A's protected records. Standalone customer-account Deposits has been removed; order-specific payment/deposit instructions remain.
- Admin staff: granular view/manage permissions on the same Admin routes. Owner-only settings and sensitive identity/credential actions must remain unavailable even if a staff role is misconfigured. Owner access to Admin requires the configured MFA path.
- Telegram chat and Mini App: unauthenticated chat, linked account, signed tracking capability and short-lived Mini App session are distinct; do not treat chat identity as website ownership until linking succeeds.

- Expected results in each case are the pass criteria. Preserve exact amounts, assets/networks and customer-safe projections; absence of silent writes, duplicate orders, false success notices and cross-account data is part of the result.

Suggested execution order

1. Confirm routing, auth gates, health/readiness, catalogs and safe test fixtures.
2. Test public reading/search/forms, then Swap and Convert guest/customer journeys and order confirmation.
3. Test account, affiliates and Telegram-linked journeys against the same test orders.
4. Test Admin views, scoped mutations, catalog/pricing, editorial tools and role boundaries.
5. Test synthetic monitoring, provider failures, email/outboxes, config sync preview and decommission checks.
6. Recheck narrow screens and keyboard paths after data-dependent views have loaded.

In-scope route and service inventory

- Public web: `/`, `/swap`, `/convert`, `/about`, `/affiliates`, `/contact`, `/privacy`, `/terms`, `/aml-kyc`, `/how-it-works`, `/user-manual`, `/market-rates`, `/crypto-pairs`, `/faq`, `/blog`, `/blog/:slug`, `/status`, `/order/:id`. Alias redirects: `/about-us`, `/affiliate-program`, `/contact-us`.
- Customer web: `/sign-in`, `/sign-up`, `/account`, `/account/orders`, `/account/orders/:id`, `/account/settings`, `/account/affiliate`, `/telegram/connect`. Explicit removed-route assertion: `/account/deposits` and corresponding account deposit/balance APIs must not expose the old feature.
- Admin web: `/admin`, `/admin/orders/:id?`, `/admin/revenue`, `/admin/customers/:id?`, `/admin/affiliates/:id?`, `/admin/payouts`, `/admin/affiliate-settings`, `/admin/providers`, `/admin/integrations`, `/admin/notification-settings`, `/admin/currencies`, `/admin/pricing`, `/admin/appearance`, `/admin/landing-background`, `/admin/team`, `/admin/site-content`, `/admin/blog`, `/admin/blog/new`, `/admin/blog/edit/:id`, `/admin/blog/automation`, `/admin/newsletter`; `/admin/staff` redirects to `/admin/team`.
- Telegram Mini App: Home, Exchange, Orders, order detail, Track ID, Support, Account. Telegram bot: private-chat menu, Swap wizard, Orders/Track, language/support and website account linking.
- API/runtime: `healthz/readyz`, public exchange/`quickex/config/content/blog/affiliate` endpoints, Admin order/customer/catalog/pricing/provider/monitoring/content/team routes, webhook handlers, email/outbox processing, WhiteBIT reconciliation, Telegram webhook and Mini App sessions. Admin-only `orders.xml` export is implemented.

Observability and cautions

- The six configured workflow surfaces include API Server, crypto-exchange web, Telegram Mini App web, mockup preview, API tests and API typecheck. The mockup preview is design tooling, not a customer-facing product flow. For a manual smoke check confirm the three runtime app surfaces open and API `/healthz` and `/readyz` return healthy responses in a ready test environment; an automated tests workflow failing is not itself evidence a manual scenario failed.
- Published Site Content and scheduled blog automation have separate draft/preview/publish or scheduled-run boundaries. Do not publish public-facing content or send a newsletter to real subscribers when exercising these tests.
- Admin Customer Deposits toggles on exact crypto routes are for Swap order funding; they are not the removed account Deposits panel. Historical standalone addresses/ledger must remain intact pending an audited retirement decision.
- Source notes following cases identify implementation/tests inspected. They are pointers for diagnosing a failed human test, not steps a tester must run.

Contents

01	Public pages, content and discovery
02	Swap, Convert, confirmation and tracking
03	Customer accounts, affiliates and linking
04	Admin operations and reporting
05	Admin catalog, pricing and provider controls
06	Admin content, editorial, branding and team
07	Telegram bot and Telegram Mini App
08	Operational APIs, monitoring and integrations
END	Recording and sign-off

Public pages, content and discovery

Inventory (live public React routes; exchange wizard intentionally excluded)

Routing is in `artifacts/crypto-exchange-widget/src/App.tsx`; public registry is `src/lib/public-page-registry.ts`. `/` is `ExchangePage`, `/swap` and `/convert` are configured widget pages (excluded here). The inspected live non-wizard modules are:

- `/about` → `AboutPage`: managed/published hero (or defaults), optional hero image, four numbered “What makes QuickXchange different?” value cards. Disabled managed content renders `NotFound`. No user controls.
- `/affiliates` → `AffiliatesPage`: managed/published hero and three numbered “How Does It Work?” cards (“Ask people to Join”, “Engage to Exchange”, “Earn revenue”). Disabled page → `NotFound`. No controls.
- `/contact` → `ContactUsPage`: resource links “Check your swap status” (`/status`), “How to use our service” (`/how-it-works`), a non-clickable FAQ card; Telegram external link and `mailto`; Contact Form fields Name (max 120), Email (max 320), Message (max 5000), `Send`; newsletter Email address and `Subscribe`. Managed body/sections may replace the normal content while preserving children; managed primary/secondary anchors can appear. Legacy `/about-us`, `/affiliate-program`, `/contact-us` redirect to canonical paths.
- `/privacy` → `PrivacyPolicyPage`: hero plus fixed number of privacy legal cards, managed title/subtitle/SEO/hero/section copy; disabled → `NotFound`. No controls.
- `/terms` → `TermsConditionsPage`: same structure using terms cards; disabled → `NotFound`. No controls.
- `/aml-kyc` → `AmlKycPage`: hero; default AML framework and KYC requirements cards (identity document, address proof, liveness, possible source-of-funds) or managed body/sections; disabled → `NotFound`. No controls.
- `/how-it-works` → `HowItWorksPage`: explanatory Swap and Convert animated mockups, actual CTAs `Start Swap` → `/swap#exchange-widget`, `Start Convert` → `/convert#exchange-widget`; Telegram Bot instructional six-step list and external `Open QuickXchange Bot` (target blank). Mockup is presentation-only, not an interactive wizard. Responsive one-column layout on narrow screens; motion stops under reduced-motion.
- `/user-manual` → `UserManualPage`: long documentation page with sticky/scrolling Contents anchors, links to Home, Swap, Convert, Track an Order `/status`, Account `/account`, My Orders `/account/orders`, Telegram/email support. Three image figures open enlarged-image dialog; dialog controls `Fullscreen`, `close` (Escape and backdrop click); instructional video is explicitly a non-interactive placeholder (“A guided video will appear here...”). Responsive TOC/content and figures.
- `/market-rates` → `MarketRatesPage`: CoinGecko market query (250); `Search coins...`; table columns `#`, `Asset`, `Price`, `24h Change`, `Market Cap` (hidden below sm), `Volume (24h)` (hidden below md). Every data row is keyboard/click link to `/convert?asset=SYMBOL`; loading skeletons, provider error panel, and exact no-match text. “Updated ...” and “Data may be delayed” staleness badge.
- `/crypto-pairs` → `CryptoPairsPage`: Quickex config/routes plus market data. Three category cards: “Popular Crypto Pairs”, “Popular Stablecoin Pairs”, “Trending Crypto Pairs”, each 3 rows/page with `Previous` pairs/`Next` pairs controls and `Convert` buttons. Main filters `Send` and `Receive` selects (default “All Assets”), `Search pairs`, e.g. `BTC/ETH...`; table columns `Pair`, `Rate / Price`, `24h Change`, `Status`, `Convert`; 50 rows/page. Pair convert preserves symbols/networks via selection state and navigates to `Convert`. Loading skeletons, API error “Unable to load pairs. Please try again later.”, no-route “No conversion routes currently available.” and filter-empty “No conversion routes match your filters.”; absent market quote is `–` / “Live data unavailable”. Mobile cards stack; table scrolls horizontally.

- `/faq` → `FaqPage`: 17 FAQ accordion items, first open initially; each trigger toggles itself independently, aria-expanded and inert collapsed region. Answers contain operational caveats and links to `/status` and `/contact`.
- `/blog` → `BlogPage`: API categories/articles, query URL state `?category=`, `?tag=`, `?search=`, `?page=`; category tabs "Latest" plus API category names; Search intelligence... form and clear-search X; featured story/latest briefs/more grid; article links `/blog/:slug`; CTA "Open Convert Desk" (`/convert`), "Manual Swap" (`/swap`), "Supported Markets" (`/crypto-pairs`), "Platform FAQ" (`/faq`); pagination "Newer"/"Older". Loading, API error "Failed to load articles. Please refresh the page.", empty "No intelligence found", search-specific explanation and Clear all filters. Toolbar is sticky and horizontally scrollable on mobile.
- `/blog/:slug` → `BlogDetailPage`: loading/error ("Intelligence briefing not found or has been removed." + Return to Intelligence), sanitized HTML or text body, breadcrumbs, category/title/excerpt/author/date/updated/read time, optional image/tags/citations, previous/next brief, CTA cards. Share controls: native Share where supported, Copy link with temporary "Copied", Twitter/LinkedIn/Facebook external links; share block is mobile-visible while desktop controls are hidden/side rail. Missing citations/tags/prev/next/image simply omit those sections.
- `/order/:id[?provider=manual|quickx&trackingToken=...]` → `OrderConfirmationPage`: polls the provider-specific status endpoint. Loading "Securing your order..."; 404 "Order Not Found" + Return to Exchange; status presentation Pending/Confirming/Processing/Completed or Cancelled/Expired/Failed, progress steps (manual Created/Detected/Processing/Done; Convert Created/Confirming/Processing/Done), refresh warning. Exchange Summary shows You Send/You Receive, date, copyable Order ID, and manual rate. Manual payment details can expose Mark as Paid through PaymentDetailsCard; address provisioning says "Generating your deposit address"; unresolved says "Deposit address unavailable", "Do not send funds...", Check again; deposit flow uses Show Deposit Details, QR/address/memo copy buttons and hide button; links Track this order to `/status?...` and Start another conversion to `/`. Terminal orders disable payment action. Responsive cards stack and IDs truncate.

Human manual tests

PUB-01 - Verify informational pages and disabled behavior

Role/route: Guest; `/about`, `/affiliates`, `/privacy`, `/terms`, `/aml-kyc`. **Preconditions:** App available; published content may or may not be configured. **Steps:**

1. Open each canonical URL directly and refresh.
2. Confirm PublicShell/header/footer render and page title/hero copy is visible.
3. On About, verify four numbered value cards; on Affiliates, three numbered cards.
4. On Privacy and Terms, scroll every legal card; on AML/KYC, verify AML and KYC copy and the four typical verification requirements.
5. If an admin disables a page, reload that URL.

Expected (observable): Defaults appear when no managed copy exists; configured title/subtitle/sections/images replace corresponding defaults; disabled content shows NotFound rather than a blank page. These pages have no actionable forms/buttons. On mobile, hero and cards remain readable and stack without horizontal clipping.

PUB-02 - Contact form required/format validation

Role/route: Guest; `/contact`. **Preconditions:** Contact page is visible. **Steps:**

1. Submit Contact Form with all three fields empty.
2. Enter whitespace-only Name or Message and an invalid Email such as `x@`; click Send.
3. Enter valid Name, Email, and Message; submit.
4. During submission, observe the button.

Expected (observable): Client validation displays exactly Please enter a valid name, email address, and message. and does not call submission. Valid values are trimmed before submission; pending Send is disabled and shows spinner; success clears all fields and says Message sent successfully. Our support team will get back to you soon. . A server failure says Your message could not be sent. Please try again later. . At 120/320/5000-character limits inputs stop accepting further characters; contact panel stacks on mobile.

PUB-03 - Newsletter validation and submission

Role/route: Guest; /contact . **Preconditions:** Scroll to “Stay in touch”. **Steps:**

1. Submit empty or malformed Email address .
2. Enter a valid email with mixed case and spaces and click Subscribe .
3. Repeat with the API unavailable.

Expected (observable): Invalid input shows Please enter a valid email address. without a request. Valid input is trimmed/lowercased; pending Subscribe is disabled/spinner; success clears input and says Thanks – you’re subscribed to QuickXchange updates. ; failure says We could not subscribe you right now. Please try again later. . The form changes from row to column on narrow screens.

PUB-04 - Contact resource and support links

Role/route: Guest; /contact . **Steps:** Click Check your swap status , then go back and click How to use our service ; inspect the FAQ card; click Telegram and email support. **Expected (observable):** First navigates /status , second /how-it-works ; FAQ card is visibly informational and has no click behavior; Telegram opens the configured external Telegram URL in a new tab and email invokes mailto: . On mobile, resource cards remain full-width and readable.

PUB-05 - How-it-works CTAs and responsive animation

Role/route: Guest; /how-it-works . **Steps:** Confirm Swap mockup cycles through its six presentation states; click Start Swap . Return, click Start Convert ; return and click Open QuickXchange Bot . **Expected (observable):** Mockups show only visual examples (“Choose assets”, “Enter amount”, “Receiving Details”, review, funds/deposit, completion) and do not accept input. CTAs navigate exactly to /swap#exchange-widget and /convert#exchange-widget ; bot opens configured external target. With reduced motion, mockup does not auto-cycle. At mobile width sections become one column and CTAs remain usable.

PUB-06 - User manual contents and internal links

Role/route: Guest; /user-manual . **Steps:** Click each Contents item (Introduction , Swap vs. Convert , Starting an Exchange , Selecting Assets & Networks , Reviewing Your Quote , Entering Details , Terms & Submission , Funding Your Order , Tracking Your Order , Account Features , History & Notifications , Affiliate Program , Safety & Troubleshooting , Common Questions , Support). Click Home, Swap, Convert, Track an Order, Account, and My Orders links. **Expected (observable):** Each anchor scrolls to its matching section; links resolve to / , /swap , /convert , /status , /account , /account/orders . Copy explicitly warns about exact amount, wrong network, missing memo/tag, expired orders and irreversible addresses; do not test those warnings as implemented validation. Video controls are decorative/non-interactive.

PUB-07 - User manual image lightbox and mobile close paths

Role/route: Guest; /user-manual . **Steps:** Click Swap interface image, inspect dialog and hotspots, click Fullscreen , exit fullscreen if available, click close X; repeat for Convert and Tracking images; reopen and press Escape; reopen and click backdrop. **Expected (observable):** Dialog has enlarged image, title, caption, Fullscreen , and Close enlarged image ; fullscreen is attempted only when browser supports it; X, Escape, and backdrop close. Missing fullscreen support does not break close. Images are lazy-loaded and remain contained on mobile.

PUB-08 - Market-rate loading, searching, and conversion navigation

Role/route: Guest; /market-rates . **Steps:** Load with network throttled and observe skeleton rows; restore network. Search by a displayed coin name, then symbol; enter an unmatched string. Click a result row; repeat using keyboard focus + Enter and Space. **Expected (observable):** Initial table shows 15 skeleton rows; data shows rank/asset/price/24h change, with Market Cap hidden below small breakpoint and Volume hidden below medium

breakpoint. Search filters case-insensitively; empty result says No cryptocurrencies found matching "<query>". Clicking, Enter, or Space navigates /convert?asset=UPPERCASE_SYMBOL; rows expose aria-label="Convert <name>". Provider failure without prior data shows Market data temporarily unavailable and explanatory text. Stale/refetch failure can show Data may be delayed.

PUB-09 - Crypto-pairs category cards and pair conversion

Role/route: Guest; /crypto-pairs. **Preconditions:** API returns at least four routes. **Steps:** Wait for category cards; inspect network labels and live reference values. In each category, click Next pairs then Previous pairs; click a row's Convert. **Expected (observable):** Three category cards show up to three pairs per page and page indicator n / total; first Previous and final Next are disabled. Each pair displays source/destination symbols, networks, rate or Live data unavailable, optional percent change. Convert navigates to Convert with source/destination asset and exact networks preserved. Empty category says No pairs available for this category right now.

PUB-10 - Crypto-pairs filters, search, pagination, and API failures

Role/route: Guest; /crypto-pairs. **Steps:** Select a concrete Send asset and Receive asset; type a symbol/name in Search pairs, e.g. BTC/ETH...; enter nonsense; if >50 results use table page controls; throttle initial API. **Expected (observable):** Selects start at All Assets; filters combine with search and reset table to page 1; unmatched table says No conversion routes match your filters.; pagination displays only up to 50 and next/previous changes page. Initial loading shows three 340px skeleton cards; config/routes error says Unable to load pairs. Please try again later.; no resolved routes says No conversion routes currently available.. Mobile filters stack and the wide table scrolls rather than clipping.

PUB-11 - FAQ accordion and keyboard semantics

Role/route: Guest; /faq. **Steps:** Confirm first question is open; click it; click another question; click that question again; use Tab/Enter/Space on triggers; activate Track an Order and Contact Us answer links. **Expected (observable):** First answer is initially visible; clicking open item collapses it, clicking another opens that one (previous closes); aria-expanded and answer region visibility/inert state track the state. Links navigate /status and /contact. Accordion remains usable at mobile width.

PUB-12 - Blog loading, category/search URL state, and empty results

Role/route: Guest; /blog. **Steps:** Throttle article API; restore and click Latest and an API category; submit blank search, then Search intelligence... with a keyword; click X clear; request a nonsense search/category. **Expected (observable):** Loading uses content placeholders; API failure says Failed to load articles. Please refresh the page.. Category/search updates URL with page 1 and reloads content/title; blank search removes search; X removes search, returns page 1, and refocuses input. Empty state is No intelligence found; keyword variant says it could not find a match; filtered state exposes Clear all filters. Toolbar is horizontally scrollable/sticky on mobile.

PUB-13 - Blog article navigation and pagination

Role/route: Guest; /blog with >=14 articles. **Steps:** Open featured story, a Latest Brief, and a More Intelligence card; return and click Older, then Newer; inspect first/last page disabled states; click CTA cards. **Expected (observable):** Cards navigate /blog/<slug>; first page Newer disabled and final page Older disabled; page query changes and scroll returns to top. CTAs navigate /convert, /swap, /crypto-pairs, /faq. Featured layout appears only unfiltered page 1; filtered results use list/grid.

PUB-14 - Blog detail success, sanitized content, and missing slug

Role/route: Guest; /blog/:slug. **Steps:** Open an existing slug; inspect breadcrumb/category/title/excerpt/metadata, body, image/tags/citations; try a slug that does not exist; use Return to Intelligence. **Expected (observable):** Loading placeholder precedes article; HTML body is rendered after sanitization, plain body preserves whitespace; optional sections are absent when data is absent; citation links open external source in a new tab. Missing/error shows Intelligence briefing not found or has been removed. and Return to Intelligence, which goes /blog. No imaginary retry control is expected.

PUB-15 - Blog sharing on desktop and mobile

Role/route: Guest; existing /blog/:slug. **Steps:** On desktop click Copy link; if browser supports Web Share click Share and cancel; click Twitter, LinkedIn, Facebook; resize to mobile and repeat Copy link. **Expected (observable):** Copy button briefly changes icon and shows Copied when clipboard succeeds; native Share exists only where navigator.share exists and cancellation is ignored; social links open target share URLs in new tabs. Desktop share

controls are in metadata/side rail; mobile share block appears below content. Clipboard rejection has no visible error by implementation.

PUB-16 - Order lookup loading and not-found outcomes

Role/route: Guest; `/order/<id>` with manual and `provider=quickex` variants, optionally `trackingToken`. **Steps:** Open a valid order URL with slow status response; open an invalid ID; open a valid ID with the wrong/missing token; observe through refresh/poll intervals. **Expected (observable):** Initial state says `Securing your order...` and `Please wait while we confirm details...`. 404 renders `Order Not Found`, localized lookup error text, and `Return to Exchange to /`. Manual and Quickex call different status APIs; token is forwarded when present. Polling refreshes while non-terminal and stops for terminal statuses. Non-404 failures show the localized refresh-failed state or live-update warning, not fabricated order data.

PUB-17 - Manual order payment actions and terminal safeguards

Role/route: Guest; valid manual `/order/:id`. **Steps:** For a pending order with payment details, inspect `Payment Details`; click `Mark as Paid`; refresh status. For a terminal/failed/expired/cancelled order, inspect the same area. For provisioning and unresolved deposit states, inspect controls. **Expected (observable):** Pending shows route-specific payment instructions and `Mark as Paid`; clicking submits once, disables while pending, then invalidates status. Terminal status disables payment actions. Provisioning says `Generating your deposit address` and advises keeping page open; unresolved says `Deposit address unavailable`, warns not to send funds, shows order ID and `Check again`; `Check again` refetches and disables while fetching.

PUB-18 - Crypto deposit details, copy controls, tracking and mobile layout

Role/route: Guest; valid order with `depositAddress` (manual or Quickex). **Steps:** Click `Show Deposit Details`; inspect exact amount, address, QR and optional memo/tag; click copy order ID, deposit address, and memo; click `Hide Deposit Details`; click `Track this order`; click `Start another conversion`. **Expected (observable):** Details expand to QR/address/memo and can be hidden with the implemented secondary button; clipboard buttons briefly show the copied state. Tracking goes to `/status?order=<id>` and preserves `trackingToken` when present; `Start another conversion` goes `/`. Completed/failed/expired/cancelled orders do not present an actionable deposit path. Narrow screens stack summary/payment blocks, preserve exact values, and truncate long IDs without breaking controls.

Source note

`artifacts/crypto-exchange-widget/src/App.tsx`; `src/lib/public-page-registry.ts`; `src/pages/public-info-pages.tsx`:126-582; `src/pages/market-rates.tsx`:10-168; `src/pages/crypto-pairs.tsx`:1-613; `src/pages/faq.tsx`:27-185; `src/pages/how-it-works.tsx`:470-704; `src/pages/user-manual.tsx`:148-621; `src/pages/blog.tsx`:9-464; `src/pages/blog-detail.tsx`:12-414; `src/pages/order-confirmation.tsx`:39-607.

Swap, Convert, confirmation and tracking

Inventory (public journey)

- `artifacts/crypto-exchange-widget/src/App.tsx` owns `/order/:id` and `/status`, provider selection, polling, status presentation, tracking lookup, and public routes. `App.tsx:1816-1828` aliases mode tabs at runtime to `button-mode-select-instant` (Convert) and `button-mode-select-manual` (Swap).
- `artifacts/crypto-exchange-widget/src/components/exchange-surface.tsx` is the manual Swap form: quote lifecycle, mode-independent selectors, Step 1/Step 2, payment/settlement fields, refund rules, guest vs signed-in email, submit/uncertain handling.
- `artifacts/crypto-exchange-widget/src/components/quickex-convert-widget.tsx` is Convert. Config/direct-pair filtering and URL hydration are at `:160-329`; debounced quote and expiry at `:331-384`; submit/validation/order redirect/uncertain outcome at `:398-469`; rendered controls at `:508-866`.
- `artifacts/crypto-exchange-widget/src/pages/order-confirmation.tsx` owns both `/order/:id?provider=manual|quickex`, status polling, status labels, deposit/payment details, QR/copy/mark-paid, tracking and start-another actions.
- `artifacts/crypto-exchange-widget/src/components/order-completion.tsx` renders only completed receipts and configured Trustpilot review, including PDF/Print behavior.
- E2E evidence: `e2e/live-quote.spec.ts`; `e2e/completion-receipt.spec.ts`; responsive coverage `e2e/responsive-layout.spec.ts`; selector coverage `e2e/web-selector-logo-ui.spec.ts`.

SW-01 - Guest Swap, quote to manual order

Role/route: Guest visitor; `/` (manual Swap active). **Preconditions:** Manual config has a directed pair and quote API returns an unexpired quote; use `?__e2eGuest=1` when reproducing E2E guest mode. **Steps:**

1. Confirm mode button `button-mode-select-manual` has `aria-pressed="true"`; locate amount `input-amount`, source `select-from-asset`, target `select-to-asset` and read-only `input-receive-amount`.
2. Click `select-from-asset`; use its search textbox (`search-select-from-asset`) and an option (`option-select-from-asset-<id>`); repeat target with `select-to-asset`. Select only a pair shown by config.
3. Fill `input-amount` with a positive value such as `100`; wait for quote. Observe receive amount, rate/fee, min/max and expiry countdown; `button-swap-continue` becomes enabled only with a current quote.
4. Click `button-swap-continue`; expect `swap-step-wallets/swap-wallet-quote-summary`, quote amount/rate and Step 2 progress. Destination `input-destination-address` is required for crypto payout. Fill it (and `input-destination-memo` if offered).
5. Fill guest `input-customer-email`; fill any required dynamic payment cards `input-detail-<field.key>` (cards are `swap-detail-card-<field.key>`). Optional cards visibly say (Optional).
6. If shown, fill `input-refund-address`; if the source network requires a memo/tag, the revealed `input-refund-memo` becomes required. Check `#swap-terms`.
7. Click `swap-button-submit` and wait for redirect.

Expected (observable): Exactly one order request is made; URL is `/order/<id>?provider=manual&trackingToken=<token>`; heading `order-created` is visible. Submitted body includes `type: manual`, selected source/target, amount, destination, guest email, settlement details and `quoteId`; stale/invalid pair cannot proceed. Evidence: `exchange-surface.tsx:1026-1081,1097-1100,1147-1241`; `live-quote.spec.ts:681-730`.

SW-02 - Signed-in Swap without duplicate contact fields

Role/route: Authenticated customer; /. **Preconditions:** Clerk/session customer is present; manual route and quote available. **Steps:**

1. Switch to `button-mode-select-manual`; select a source/target pair and fill `input-amount` until `button-swap-continue` enables.
2. Continue and inspect `swap-step-wallets`: complete destination, dynamic settlement fields and terms.
3. Verify `input-customer-email` is prefilled with the signed-in verified email and disabled; there is no second editable guest contact field. Submit with `swap-button-submit`.

Expected: Request uses authenticated customer context and does not duplicate checkout contact fields; order redirects to manual confirmation. The account email is not separately required from guest input and is omitted from the guest email field in the request. This is the intended regression surface noted by Source: `exchange-surface.tsx:1218-1225,1545-1707`; `e2e/customer-account.spec.ts` (customer order surfaces).

SW-03 - Swap quote invalidation, min/max and API failure

Role/route: Guest; /. **Preconditions:** Quote fixture supports success, failure and bounds. **Steps:**

1. Fill an invalid amount (0, negative, non-numeric or blank), then a value below `minAmount` or above `maxAmount`; observe `input-receive-amount` and Continue.
2. Change source, target, amount or route after a quote appears; immediately inspect Continue and request count.
3. Make quote endpoint fail, then restore it and edit amount or use the visible refresh/retry affordance.
4. Attempt pressing Enter or clicking Continue while no current quote exists.

Expected: Receive remains blank or quote error says rate unavailable; Continue is disabled and no order request occurs. Any changing quote key clears prior quote, preventing use of a quote for another route; failed quote shows user-facing unavailable text and remains non-submittable. Evidence: `exchange-surface.tsx:952-1081,1097-1145`; `live-quote.spec.ts:732-~930`.

SW-04 - Swap payment/settlement fields across responsive widths

Role/route: Guest; /; desktop, 390x844 and 320x720. **Preconditions:** Fiat/payment route returns `standardFiatFields`. **Steps:**

1. At each viewport select the configured route, fill `input-amount`, click `button-swap-continue`.
2. Assert each expected `[data-testid="swap-detail-card-"]` appears in config order; inspect `input-detail-name`, `input-detail-bank_detail`, `input-detail-bank_name`, `input-detail-payment_description`, `input-detail-telegram_or_whatsapp` (actual keys vary by config).
3. Check required markers and (Optional) labels. Tab from first card through each field to `input-customer-email`; scroll to `#swap-terms` and submit control.

Expected: Required inputs have HTML `required` and a `.required-field-mark`; optional inputs do not. No horizontal document overflow at 1440/390/320; cards and submit remain reachable, with no disappearing payment fields during route changes. Evidence: `live-quote.spec.ts:639-673`; `e2e/responsive-layout.spec.ts`; ### SW-05 - Swap crypto refund memo rules and invalid memo **Role/route:** Guest; /. **Preconditions:** Crypto-to-fiat/manual source (e.g., XRP) whose network can require a tag. **Steps:**

1. Select crypto source and fiat target, quote and continue.
2. Verify `input-refund-address` is optional; initially `input-refund-memo` is hidden/disabled.
3. Fill a refund address. Verify memo appears and becomes required only when source `requiresMemo`.
4. Enter `not-a-tag`, check `#swap-terms`, click `swap-button-submit`; then replace with valid `12345` and submit.

Expected: First submission is blocked with Check that the refund memo or tag matches the selected source network. and no order is created. Valid memo submits it in `refundMemo`; refund fields are omitted when blank. Evidence: `live-quote.spec.ts:935-1065`; `exchange-surface.tsx:1631-1669,1170-1181`.

SW-06 - Swap fiat-to-crypto does not require refund details

Role/route: Guest; /. **Preconditions:** USD/EUR fiat source to crypto target route. **Steps:**

1. Select fiat source and crypto target, fill amount, continue.
2. Confirm destination field exists; confirm refund address/memo controls are absent for fiat-to-crypto.
3. Fill destination, guest email and terms; click `swap-button-submit`.

Expected: One successful manual order request contains destination and contact but no `refundAddress` / `refundMemo`; no refund validation error. Evidence: `live-quote.spec.ts:1072-1171`; `exchange-surface.tsx:1631`.

SW-07 - Swap back navigation preserves or refreshes correctly

Role/route: Guest; /. **Preconditions:** Current quote is valid. **Steps:**

1. Continue to Step 2; fill destination/email and any details.
2. Click `button-swap-back`, change source/target or amount, and observe Step 1.
3. Return to Step 2 with `button-swap-continue`.

Expected: Back returns to quote without creating an order; changed route/amount invalidates old quote and requires a fresh quote. Unchanged valid quote retains valid summary and data where implemented. A quote expiry while on Step 2 shows refresh state rather than allowing submit. Source: `exchange-surface.tsx:989-997,1147-1167,1729-1731`; `live-quote.spec.ts:697-711`.

CV-01 - Guest Convert, directed quote and wallet order

Role/route: Guest; /, choose Convert via `button-mode-select-instant`. **Preconditions:** `/api/quickex/config` has `signedOrders:true`, instruments and directed pair (e.g., BTC Bitcoin → USDT TRC20). **Steps:**

1. Click `button-mode-select-instant`; confirm `convert-select-from-asset` and `convert-select-to-asset` have configured `data-value` values and no Swap payment selectors (`select-payment-method`, `select-payout-method`, `input-order-note`).
2. Open source selector; use `search-convert-from-asset`, category buttons such as `filter-convert-from-asset-all` / `-crypto`, and `option-convert-from-asset-btc-bitcoin`; repeat target as needed. Escape closes it.
3. Fill `convert-input-amount` (1); read-only `convert-input-receive-amount` populates after debounced quote. Choose `convert-rate-floating` or `convert-rate-fixed` and wait for the corresponding quote.
4. Click `convert-button-continue`; verify `convert-step-wallets`, `convert-wallet-quote-summary`, `convert-summary-from-logo`, `convert-summary-to-logo`, and focus on `convert-input-destination-address`.
5. Fill `convert-input-destination-address`, optional `convert-input-destination-memo`, `convert-input-email`; optionally fill `convert-input-refund-address` (then optional/conditional `convert-input-refund-memo`). Check `#convert-terms`.
6. Click `convert-button-submit`.

Expected: Request goes to Quickex quote/create endpoints with `type: instant`, selected networks, amount, `quoteId`, chosen `rateMode`, destination/memo/email. Redirect is `/order/<id>?provider=quickex&trackingToken=<token>`; heading `order-created` and status `order-confirmation` (e.g. AWAITING FUNDS) appear; quote form is gone and no cancel button is shown. Evidence: `quickex-convert-widget.tsx:331-369,398-440,546-866`; `live-quote.spec.ts:1176-1387`.

CV-02 - Convert fixed/floating rates, reverse route and stale defaults

Role/route: Guest; /. **Preconditions:** Config includes BTC→USDT and optionally reverse USDT→BTC; default pair may be stale. **Steps:**

1. Enter a valid amount and record receive value under `convert-rate-floating`.
2. Click `convert-rate-fixed`; wait for a new quote and compare receive/rate; switch back to floating and verify a fresh request.
3. Click `convert-button-swap-assets` when reverse pair is configured; inspect source/target values and quote. Repeat with a one-way pair where reverse is unavailable.
4. Reload or let config refetch after selecting a non-default pair.

Expected: Rate mode is reflected in request and receive amount; no old response overwrites a newer request. Reverse button is enabled only when a directed reverse pair exists; otherwise it is disabled with "Reverse unavailable" accessibility/title. Invalid/stale configured default falls back to first valid directed pair; user selection survives config refetch. Evidence: `quickex-convert-widget.tsx:171-205,331-369,386-391`; `live-quote.spec.ts:433-575,1325-1337`.

CV-03 - Convert wallet validation, required fields and back

Role/route: Guest; /. **Preconditions:** Valid current Quickex quote; validation endpoint can return valid/invalid. **Steps:**

1. Continue with blank destination; inspect submit and native required behavior.
2. Fill email but leave destination blank; then fill destination and optional memo. Leave terms unchecked and inspect `convert-button-submit`.
3. Submit with invalid wallet response; then correct wallet and retry. Use `convert-button-back`, return forward and inspect all values/focus.

Expected: Submit is disabled until destination/email/terms/current quote are present. Invalid destination yields `convert.checkWallets` notice and no create-order request. Back returns to quote and forward refocuses destination while retaining destination memo/email. Evidence: `quickex-convert-widget.tsx:398-420,666-735,739-866`; `live-quote.spec.ts:1341-1364`.

CV-04 - Convert quote expiry and refresh

Role/route: Guest; /. **Preconditions:** Quote fixture expires quickly or wait until `expiresAt`. **Steps:**

1. Obtain quote and click Continue before expiry; observe wallet step.
2. Wait past expiry or force clock beyond `expiresAt`; inspect `convert-expired-quote`.
3. Click its `Refresh quote` button; wait for a new quote and return to wallets.

Expected: Expired quote cannot submit. Step 2 shows the refresh notice and `Refresh quote`; clicking returns to quote, increments refresh and obtains a current quote. Source: `quickex-convert-widget.tsx:371-384,505-506,722-736`; `order-confirmation.tsx:60-89` for post-order polling.

CV-05 - Convert unavailable/no routes and Swap fallback

Role/route: Visitor; /. **Preconditions:** Quickex config error, `signedOrders:false`, empty instruments or no target route. **Steps:**

1. Select Convert and inspect `convert-unavailable`.
2. Click `button-retry-convert`; if still unavailable click `button-use-swap`.
3. Separately select a valid source with zero receive routes and inspect the inline error; click `button-retry-receive-options`.

Expected: Copy states no quote/order will be created; `button-retry-convert` and `button-use-swap` are visible. Fallback activates Swap (`button-mode-select-manual` pressed and `form-exchange` visible). No-route state shows

refresh currencies and Continue cannot create a quote/order. Evidence: quickex-convert-widget.tsx:471-503,623-637 ; live-quote.spec.ts:1389-1417 .

CV-06 - Convert uncertain create outcome / same request recovery

Role/route: Guest; /. **Preconditions:** Address validation succeeds, create-order responds network failure, outcomeUnknown:true, or code:'verification-required'. **Steps:**

1. Complete Convert wallet step and click `convert-button-submit` once.
2. Observe notice and the mode tabs/button state; do not switch modes or blindly create a second order.
3. If response includes an `orderId`, reload/follow the order URL; otherwise click the submit control labeled `Check same request` only after confirming the original attempt result.

Expected: If known order ID accompanies uncertain error, app redirects to `/order/<id>?provider=quickex`. Otherwise it shows Convert uncertain messaging, blocks Swap tab (`convert.switchBlocked`), changes submit label to `Check same request`, and does not reset request identity; ordinary definitive failures reset request ID/terms and show provider error. Source: quickex-convert-widget.tsx:413-467,513-525,856-865 ; analogous Swap handling `exchange-surface.tsx:1240-1294` .

OR-01 - Manual order confirmation, payment, mark-paid and tracking

Role/route: Guest order owner; `/order/<id>?provider=manual&trackingToken=<token>`. **Preconditions:** Created manual order returned with payment/crypto deposit details. **Steps:**

1. Verify `heading-order-created`, `status-order-confirmation`, Order ID `text-order-id`, exchange summary (You Send/You Receive), and status timeline (Created/Detected/Processing/Done).
2. For crypto deposit click `Show Deposit Details`; inspect `text-deposit-amount`, network warning, QR, Deposit Address, optional memo and copy buttons `button-copy-deposit` / `button-copy-deposit-memo`; click `Hide QR`.
3. For fiat/manual payment inspect `order-confirmation-payment-card`, Payment Details and dynamic payment details. Click its Mark paid action (if present); expect status refetch and action disabled after terminal state.
4. Click `button-track-this-order`; verify `/status?order=<id>&trackingToken=<token>`. Use `COPY ORDER ID` (`button-copy-order-id`) and `START ANOTHER CONVERSION`.

Expected: Exact amount/network warning is visible; copy gives transient check state; tracking retains capability token. Payment controls are disabled for failed/refunded/completed orders. Source: `order-confirmation.tsx:151-233,337-426,428-575,584-602` ; `App.tsx:1236-1519` .

OR-02 - Convert confirmation and deposit lifecycle

Role/route: Guest; `/order/<id>?provider=quickex&trackingToken=<token>`. **Preconditions:** Quickex order status starts `awaiting deposit`, then transitions confirming/processing/completed. **Steps:**

1. Open URL and verify status label, Order ID, Convert exchange summary and `order-confirmation-payment-card` if deposit address exists.
2. Click `Show Deposit Details`; verify `Send Exactly`, network, QR, address/memo and warning. Copy address.
3. Change fixture/status to confirming, processing, then completed; reload or wait for polling (3 seconds until terminal).

Expected: Convert labels follow `convertOrderStatusLabel`; status/timeline progresses and deposit action disappears/locks after halted/completed. Terminal status stops polling. Source: `order-confirmation.tsx:79-89,151-215,428-575` ; `App.tsx:1252-1265` .

OR-03 - Status/tracking lookup, missing capability and not-found

Role/route: Guest or signed-in visitor; `/status`. **Preconditions:** Existing order ID with valid token; separately test unknown ID and wrong/missing token. **Steps:**

1. Enter ID in `input-order-search`, optionally preserve `trackingToken`, submit/search.
2. Open a valid result and inspect `card-order-status-<id>`, `order-status-timeline`, status badge and refresh warning.
3. Repeat with unknown ID and with wrong/missing token; open a terminal failed/refunded/expired order.

Expected: Valid lookup shows customer-safe status and capability-scoped data. 404 shows “Order Not Found” and Return to Exchange; unauthorized capability does not expose order details. Halted status shows “Refunded”, “Expired”, “Failed” or “Cancelled”, no active deposit action and no progress timeline where halted. Source: `App.tsx:1292-1325,1393-1519`; `order-confirmation.tsx:91-147`.

OR-04 - Pending/unresolved deposit address safety

Role/route: Public order owner; manual confirmation URL. **Preconditions:** Order has `fundingStatus: 'provisioning'` or `'unresolved'` and no deposit address. **Steps:**

1. Open order and inspect payment card. For provisioning wait/poll; for unresolved click `Check again`.
2. Ensure no address, QR or copy control is available while unresolved.

Expected: Provisioning says “Generating your deposit address — Keep this page open. Your address will appear here automatically.” Unresolved says “Deposit address unavailable”, “Do not send funds for this order. Contact support and provide order ID ...”; Check again is available and disabled while fetching. Source: `order-confirmation.tsx:171-191,452-487`.

RC-01 - Completed Swap receipt and print/download

Role/route: Any visitor with completed manual order; `/order/<id>?provider=manual`. **Preconditions:** Status is `completed`; receipt has fee and manual Step 2 details; Trustpilot URL is valid HTTPS Trustpilot. **Steps:**

1. Reload completed order and locate `order-completion-section`; verify kicker `TRANSACTION COMPLETE`, heading `Your Swap is complete`, copy “Your invoice is ready. Keep it for your records.”
2. Verify buttons `Download PDF` (`button-download-invoice`) and `Print` (`button-print-invoice`), plus `link-trustpilot-review`; confirm no Invoice button is rendered.
3. Download and inspect suggested filename `quickxchange-invoice-<id>.pdf`; open Print and inspect popup iframe title beginning `QuickXchange invoice`.

Expected: One-page A4 PDF contains `INV0ICE`, `PAYMENT DETAILS`, Paid/Yes, exchange rate, total, manual `PAYMENT/RECEIVING INFORMATION`, settlement fields, fee and network; it excludes blockchain transaction/provider internals. Review opens new tab with target `_blank`. Source: `order-completion.tsx:69-133`; `completion-receipt.spec.ts:8-114`.

RC-02 - Completed Convert receipt, review filtering and long details

Role/route: Any visitor; completed Quickex order. **Preconditions:** Convert is completed; Trustpilot may come from site content/public notification settings or be absent; test long saved details separately. **Steps:**

1. Open completed `/order/<id>?provider=quickex`; verify `Your Convert is complete`, `Download PDF` and `Print`.
2. Download and inspect one-page A4 output: exchange BTC→USDT, no manual `PAYMENT/RECEIVING INFORMATION` and no Fee when Convert has none; no blockchain/provider internals.
3. Configure a non-Trustpilot/HTTP review URL and verify review link is absent; configure valid HTTPS Trustpilot and verify link appears.
4. For a very long saved manual Step 2 value, download PDF and inspect that it paginates and retains the final marker text.

Expected: Receipt is shown only for completed/paid state (not processing); safe Trustpilot allowlist accepts only HTTPS `trustpilot.com` subdomains; long details preserve beginning and end over multiple pages. Evidence:

order-completion.tsx:13-23,25-53,69-80; completion-receipt.spec.ts:8-15,94-114,117-150.

RS-01 - Responsive selector and form containment

Role/route: Guest; /; phone 320/390, tablet and desktop. **Preconditions:** Both mode configs loaded. **Steps:**

1. At each viewport open Swap source selector and Convert source selector. Inspect `.swap-contained-selector / .convert-contained-selector[data-state="open"]`, overlay header, list and `Close/Close Send currency` buttons.
2. Verify Swap selector headings/filters `You Send`, `All`, `Crypto`; on touch, Fiat/Payment Methods are not rendered where the shared crypto selector contract applies.
3. Search an asset, press Escape/close, switch mode, and repeat. Open Convert selector and inspect `convert-button-swap-assets` visibility while menu is open.
4. Check document scroll width and selector geometry.

Expected: Menus remain inside widget, list scrolls rather than causing page overflow, touch targets are usable, selector width stays close to trigger (Convert difference ≤ 64 px in touch evidence), swap button is hidden while selector is open and returns afterward. Evidence: `responsive-layout.spec.ts:558-718`; `live-quote.spec.ts:1292-1316,1707-1773`.

RS-02 - Mode switching, URL hydration and guest/auth state isolation

Role/route: Guest or signed-in visitor; / with optional Convert parameters `?asset=BTC&sourceNetwork=Bitcoin&dest=USDT&destNetwork=TRC20&open=source`. **Preconditions:** Directed pairs for requested assets; optionally authenticated session. **Steps:**

1. Load URL and confirm Convert source/target hydration and requested selector opening; use invalid/stale destination/network and observe fallback.
2. Make a Swap selection, switch Convert, make a Convert selection, then switch back. Reload/config-refetch during selections.
3. Repeat signed in and guest, checking contact field behavior and that one mode's state does not overwrite the other.

Expected: URL matching is case-insensitive and network-aware; invalid directed pair falls back safely, never submits an unsupported route. Both mode trees remain mounted: mode switching preserves each mode's later selection; auth context changes guest email requirements only and never leaks another customer's data. Source: `quickx-convert-widget.tsx:217-329`; `App.tsx:1816-1828`; `live-quote.spec.ts:433-575,500-545`.

Customer accounts, affiliates and linking

Inventory (read-only)

Primary implementation: `artifacts/crypto-exchange-widget/src/pages/account.tsx` (sign-in/sign-up and affiliate-code capture at lines 104-219; claim form 221-269; account dashboard 351-553; order history 554-775; account settings 776-865; customer order detail 1364-1410). `artifacts/crypto-exchange-widget/src/pages/affiliate.tsx` contains customer affiliate page (80-229), payout request/history/referrals (231-594), admin affiliate overview/directory and payout queue, and admin settings/valuation queue (617 onward, settings 1247 onward). Shared navigation/auth/theme is `artifacts/crypto-exchange-widget/src/components/customer/CustomerShell.tsx` (profile menu 91-186; sidebar permissions/nav 189-309; responsive shell 313-423). Telegram linking is `artifacts/crypto-exchange-widget/src/pages/telegram-connect.tsx`. Behavioral coverage is in `e2e/customer-account.spec.ts` and `e2e/affiliate.spec.ts`. Public affiliate navigation is defined in `artifacts/crypto-exchange-widget/src/components/public-shell.tsx`; public affiliate route is `/affiliates` (registry in `artifacts/crypto-exchange-widget/src/lib/public-page-registry.ts`). Account standalone Deposits is intentionally absent: customer E2E explicitly asserts no `/account/deposits` link and no “Deposits” dashboard stat (`e2e/customer-account.spec.ts` 324-325).

Manual scenarios

ACC-01 - Signed-out Account gate and authentication entry points

Role/route: Guest; `/account`. **Preconditions:** Clerk session is signed out. **Steps:**

1. Open `/account`.
2. Confirm the page shows the user/account icon, “Your account”, and the sign-in description.
3. Select “Sign in”, then return and select “Sign up”.

Expected (observable): Guest sees no customer dashboard, orders, profile data, claim form, or account sidebar. “Sign in” and “Sign up” navigate to `/sign-in` and `/sign-up`. While Clerk is loading, a public loading skeleton is shown. Direct `/account/orders`, `/account/settings`, and `/account/orders/<id>` access must not expose customer data; orders/settings redirect to `/sign-in`, and order detail preserves a safe `redirect_url` back to the requested path. **Negative/edge:** Invalid/external/hash-bearing `redirect_url` values are discarded rather than followed (only same-origin account paths or tokenized `/telegram/connect` are accepted). Never rely on a guest having an order history merely because they know an order ID.

ACC-02 - Sign-in/sign-up affiliate-code capture

Role/route: Guest or newly registering customer; `/sign-in`, `/sign-up`. **Preconditions:** Affiliate program/code API available. **Steps:**

1. Open the auth page and locate “Have an affiliate code?” and “Enter affiliate code”.
2. Enter a mixed-case code and select “Apply code”.
3. Observe the transient “Checking...” state and resulting “Saved” state.
4. Repeat with blank input and an invalid/unavailable code.

Expected: Input is uppercased, `maxLength` is 12, Apply is disabled when blank/saving/already saved, and success says “Affiliate code saved. It will be connected to your account after you sign in.” Invalid API response shows an alert using “This affiliate code is invalid or unavailable.” Auth continues to work independently of code capture.

Negative/edge: Whitespace-only submission does nothing. A failed capture can be retried after editing. A referral link/code must not silently attach to a different signed-in user.

ACC-03 - CustomerShell navigation, profile, appearance, and sign-out

Role/route: Signed-in customer; any customer route (`/account` , `/account/orders` , `/account/affiliate` , `/account/settings`). **Preconditions:** Authenticated Clerk user. **Steps:**

1. Confirm sidebar labels “Dashboard”, “My Orders”, “New Exchange”, “Affiliates”, “Account”, and quick action “Track Order”.
2. Select each customer link and verify route and active styling; select the brand logo to return to the exchange surface.
3. Open the profile avatar (`button-profile-menu`), inspect displayed name/email, select “Account”, and use “Light”/“Dark” under “Appearance”.
4. Reopen the profile menu and select “Sign out”.

Expected: Customer-only routes are available and active link styling tracks location. Profile menu exposes the signed-in identity, “Account”, “Appearance”, and “Sign out”; sign-out invokes Clerk and returns to the base/public surface. Theme selection persists through the app and the light/dark controls expose pressed state.

Permissions/negative: “Admin Panel” appears in customer navigation only when the server-side operator check succeeds (or explicitly reports MFA required/enrollment required); ordinary customers do not receive that link. Customer data is not made visible merely by opening the menu.

ACC-04 - Responsive customer menu and shell controls

Role/route: Signed-in customer; customer routes, especially 320–430px widths. **Preconditions:** Mobile viewport. **Steps:**

1. Select the hamburger button (`button-mobile-menu`).
2. Confirm the “Mobile navigation” drawer, brand, all permitted links, and close button are visible.
3. Press Escape, click the backdrop, and reopen/close with “Close menu”.
4. Tab repeatedly at both ends of the drawer; navigate to another route.

Expected: Drawer is a modal navigation surface, body scrolling is locked while open, focus enters the drawer and cycles within it, Escape/backdrop/close dismiss it, and focus returns to the hamburger. Route changes close it. Links have usable touch targets (E2E checks at least 44px). No page horizontal overflow is introduced. **Negative/edge:** The standalone “Deposits” link must not appear in desktop or mobile navigation. A non-owner must not see “Admin Panel” via responsive navigation.

ACC-05 - Account dashboard exchange surface and referral binding

Role/route: Signed-in customer; `/account` . **Preconditions:** Customer has zero or more orders; affiliate API may be enabled/disabled. **Steps:**

1. Confirm the greeting “Welcome back, <name>”, “Order History”, exchange widget, stats, “Recent Orders”, and “View all”.
2. Toggle the exchange mode controls “Swap”/manual and “Convert”/instant.
3. If enabled and unbound, enter an affiliate code in “Have an affiliate code?”, then select “Apply”.
4. If already bound, inspect the “Referral connected” message.

Expected: Dashboard renders the exchange surface plus “Total orders”, “Completed orders”, “Total sent”, and “Total received”. Mode switch changes the active exchange widget. Referral input uppercases and restricts input to A–Z/2–9 (up to 12); success refreshes dashboard and a bound account shows “Referral connected” and “Your affiliate attribution is secured to this account.” Program-disabled/loading/error dashboard data hides the prompt rather than blocking the account. **Negative/edge:** Exact decimal totals are calculated without floating-point loss; completed totals only include completed/paid orders, and when more than 50 exist the scope indicates the last 50.

Multiple assets show at most two then "+N multiple assets". Zero totals show an em dash. No Cancel Order control is customer-visible.

ACC-06 - Dashboard recent-order table and empty/error states

Role/route: Signed-in customer; `/account`. **Preconditions:** Orders API returns populated, empty, delayed, or failed data. **Steps:**

1. With orders, inspect columns "Exchange", "Send", "Receive", "Status", "Date", "Action".
2. Verify up to 10 recent rows and select a row's "View" action.
3. With no orders, reload and inspect the empty state.
4. Force an orders API failure and select the retry control.

Expected: Each row shows source/target asset-network identities, exact send/receive amounts, customer-facing status pill, localized date/time, and "View" linking to `/account/orders/<id>`. Empty state says "No orders"/description and offers "New Swap"; errors show "Your order history could not be loaded." and a retry. "Provider sync delayed" is surfaced when `refreshUnavailable` is true. **Negative/edge:** Long asset/network names remain readable without page overflow; mobile uses a horizontally scrollable table and the swipe hint disappears after scrolling. Internal provider, wallet, note, error, and operator fields never appear.

ACC-07 - Claim a previous order

Role/route: Signed-in customer; `/account`. **Preconditions:** Claimable private order reference, e.g. `QX-...` or `0123456789`. **Steps:**

1. Locate "Missing an order?" (localized `account.claimTitle`) and its description.
2. Enter the private reference ID in the claim input (aria label "Private order reference ID").
3. Select "Add to account".
4. Observe loading, success, and refreshed history.

Expected: Blank input keeps "Add to account" disabled. A valid claim sends exactly `{ orderId: <trimmed ID> }`, clears the input, shows "Order added to your account", invalidates/reloads order history, and the claimed order becomes visible and navigable to detail. **Negative/edge:** Whitespace-only input does nothing; malformed, nonexistent, already-owned, or unauthorized IDs show "This order could not be added. Check the ID and try again." (or server-safe equivalent), with no order disclosure. Double submission is prevented while "Adding...".

ACC-08 - Full My Orders filtering and live refresh

Role/route: Signed-in customer; `/account/orders`. **Preconditions:** Orders include instant Convert and non-instant Swap orders across statuses. **Steps:**

1. Confirm "Order History" and tabs "All", "Swap", "Convert".
2. Search by order ID, source asset, and target asset in the search field (placeholder from `customerPortal.searchPlaceholder`).
3. Change the status select among "All", "Awaiting payment", "Processing", "Completed", and "Failed".
4. Combine tab/search/status filters and clear the search.
5. Wait for refresh or refocus the window; inspect `refreshUnavailable` messaging if returned.

Expected: Filters operate client-side: `Convert` means instant orders, `Swap` means non-instant; status groups normalize pending/awaiting/created, processing, completed/paid, and failed/expired/cancelled/refunded. Matching rows retain Exchange/Send/Receive/Status/Date/Action. No results shows "No results" and its description; no search with no orders shows the empty account state. The API is polled about every 3 seconds and refetched on focus. **Negative/edge:** Case-insensitive search; combinations that match nothing do not leak excluded rows. Signed-out direct access redirects to sign-in. Mobile table scrolls as a unit without sticky/internal fields or document overflow.

ACC-09 - My Orders customer-safe detail drawer

Role/route: Signed-in customer; `/account/orders`. **Preconditions:** At least one order in each Swap/Convert category. **Steps:**

1. Select a row's "View" action.
2. Inspect the drawer sections "Order Status", "Order Information", "Exchange Details", "Additional Payment Details" and transaction information.
3. Use "Copy Order ID", "Copy Sending Address", and "Copy customer order info" where available.
4. Close with the X/close button, Escape, and outside click; repeat for Swap and Convert.

Expected: A customer-safe drawer opens without changing `/account/orders`, shows user/order ID/sending address/created time/rate as applicable, timeline, exchange identities and customer-provided step-2 fields, and supports copying. Drawer is 480px on desktop, viewport width on small screens, scrollable with footer visible, and closes through all expected dismissal controls. **Negative/security:** Never show provider, selected provider, network ID/code, address source, diagnostics, logo URL, internal status, provider reference, operator note, deposit address, or raw provider transaction fields. Clipboard text must exclude internal fields. Customer never sees Cancel Order, cancel modal, admin actions, or staff email.

ACC-10 - Standalone customer order detail and status notifications

Role/route: Signed-in customer; `/account/orders/<id>`. **Preconditions:** Order belongs to current customer and has status pending/processing/completed/failed. **Steps:**

1. Open the row link or direct detail URL.
2. Inspect "Order Status" timeline, "Exchange Details", customer-safe step-2 details, transaction/additional payment sections, and "Email status notifications".
3. Toggle the notification switch on and off; reload after each change.
4. Test a completed/failed order and an unknown ID.

Expected: Detail polls active orders and stops polling terminal statuses. Timeline translates raw statuses to customer-facing labels, exchange values remain exact, labels/values may be copied, and the notification section explicitly says "Email only", provides one switch ("Email status notifications"), and confirms "Email status notifications are on for this order" or off. Preference persists after reload through the notifications API. Unknown order shows "Order not found"; API failure shows "This order could not be loaded." with retry.

Negative/permissions: Only the owning customer can retrieve it; unauthorized/nonexistent IDs are indistinguishable as not found. No Telegram phone field/link or Telegram button is rendered in this customer order notification control. No "Back to account" or customer cancel action is required on the standalone detail surface. Unverified/missing account email should produce a safe notification error, not a false success.

ACC-11 - Account profile, preferences, security, and Clerk profile controls

Role/route: Signed-in customer; `/account/settings`. **Preconditions:** Clerk user has any combination of first/last name, primary email, and created date. **Steps:**

1. Inspect "Account", "Personal information", "First name", "Last name", "Email", and "Account created".
2. Verify missing names render "-" and email/date are not editable by this page.
3. Under "Preferences", change the "Language" selector and use "Light"/"Dark" under "Appearance".
4. In "Security", inspect the embedded Clerk "UserProfile" and use its supported account/security controls.

Expected: Page shows only the authenticated Clerk user's identity and localized created date. Language selector and theme controls visibly update/persist. Security is a separate "Security" card with its description and Clerk profile UI; account settings do not expose order/provider data. **Negative/edge:** Signed-out access redirects to `/sign-in`; blank optional names do not render "undefined". Narrow screens scroll the embedded Clerk area rather than overflowing the page. Theme changes must not reveal a Deposits section.

AFF-01 - Public Affiliate Program page and signed-out gating

Role/route: Guest or signed-in customer; public `/affiliates` and customer `/account/affiliate`. **Preconditions:** Public page content/navigation available. **Steps:**

1. Open `/affiliates` as a guest and inspect the published affiliate program content and public navigation/footer.
2. Select “Account”/sign-in if offered, then open `/account/affiliate` while signed out.
3. Select “Sign in” or “Sign up”.

Expected: Public `/affiliates` is informational and does not expose a customer balance, private referral code, commissions, payout destination, or admin controls. Signed-out `/account/affiliate` shows the affiliate program heading/description and “Sign in”/“Sign up”, not an authenticated dashboard. Affiliate links in public navigation remain available unless explicitly hidden by public content configuration. **Negative/edge:** A guest cannot infer another customer’s referral history or use customer payout APIs. Direct admin affiliate routes require admin authorization independently of public page access.

AFF-02 - Customer affiliate dashboard and privacy-safe history

Role/route: Signed-in customer; `/account/affiliate`. **Preconditions:** Program enabled; dashboard has referral code, balances, commissions, payouts, referrals. **Steps:**

1. Confirm “Affiliate Program”, “Your referral link”, and the generated `/?ref=<code>` link; select “Copy”.
2. Inspect “Total earned”, “Referred volume”, “Commission history”, “Payout history”, “Available for payout”, “Referral activity”, “Active”, and “Total”.
3. Inspect commission rows (date/type/volume/commission), payout rows (amount/status/network/wallet/requested/decision/paid/TXID), and referral rows (date/status).
4. Use pagination controls when data exceeds a page and copy wallet/TXID controls where present.

Expected: Link is origin plus base path and referral code; Copy writes exactly that URL. Exact monetary values are formatted safely (large decimal values are not rounded into scientific notation). Histories render “Exchange”/“Reversal”, payout statuses, network code, wallet, and dates; referrals expose only joined date and active/inactive status, never customer identities, Clerk IDs, emails, or private order details. Empty sections show “No commissions”, “No payout requests”, or “No referrals”; API failures show retryable error state. **Negative/edge:** Reversal amounts display negative even if API omitted the minus sign. Legacy payout destinations show safe details/instructions rather than assuming modern wallet shape. Pagination returning an empty later page backs up to the previous page.

AFF-03 - Customer affiliate payout request validation and network sheet

Role/route: Eligible signed-in affiliate; `/account/affiliate`. **Preconditions:** Program enabled, `availableUsd` and payout networks loaded; user is payout eligible. **Steps:**

1. In “Available for payout”, inspect “Reserved”, “Paid out”, “Minimum payout”.
2. Enter a valid amount in “Amount (USDT)” (or select “Max”).
3. Open “Network” / “Select a network...”; search by code/name; select `TRC20`.
4. Enter a valid “Wallet address” (minimum 10 characters).
5. Select “Submit payout request”.

Expected: Sheet appears centered in a body portal, has “Select a network...”, search, options and selection state, closes by backdrop, and resets search on reopen. Submit sends only `{ amountUsd, networkId, walletAddress }`, shows “Payout requested successfully.”, clears form and refreshes dashboard/commission/payout queries. Button is disabled while pending. **Negative/edge:** Amount must be positive/number-like and cannot exceed available balance; exceeding balance shows the amount-exceeds-balance notice. Missing network shows a select-network error; missing/short wallet is blocked by required/minLength validation. No networks or network API failure must not create a request. If below minimum eligibility, button is disabled and “below minimum” is visible. Program paused hides the form and says “Payout requests are paused” while retaining balances/history.

AFF-04 - Customer affiliate paused, attribution, and huge-number cases

Role/route: Signed-in customer; `/account/affiliate` or account referral prompt. **Preconditions:** Program response has `programEnabled: false`, or referral is already bound, or high-precision monetary strings. **Steps:**

1. Open the customer affiliate page with program paused.
2. Confirm referral link and payout request controls are absent.
3. Verify balance/history remain visible where returned.
4. Return to `/account` with a bound attribution and inspect the referral status.

Expected: Page displays “Affiliate program paused” and explains new referral codes/commission earnings are disabled; “Payout requests are paused” explains balances/history remain visible. Bound account shows “Referral connected” and no second code form. Large exact values display deterministically (E2E expects \$9,007,199,254,740,993.12 and \$123,456,789,012,345,678.99 for representative inputs). **Negative/edge:** A transient referral-capture failure is retried and can bind after authentication; an invalid code shows a safe error. Program/attribution API loading or failure must not show a fabricated link or payout balance.

AFF-05 - Admin affiliate overview, directory, privacy, and responsive navigation

Role/route: Authorized owner/operator with affiliate permissions; `/admin/affiliates`. **Preconditions:** Server authorization grants access and affiliate APIs return overview/accounts. **Steps:**

1. Open `/admin/affiliates`; inspect metric cards “Affiliates”, “Ledger entries”, “Net commission”, “Reserved payouts”, growth chart, and “Top affiliates”.
2. Choose growth periods (7/30/90 days), search affiliate code in the directory, change status (“All”, “Bound”, “Unbound”), refresh, select rows, change page size, and paginate.
3. Open an account via its code/view action and inspect its ledger; use “View Payouts”.
4. At mobile width open admin menu and inspect “Affiliates”, “Payouts”, and “Affiliate settings”.

Expected: Overview metrics/chart render, top affiliate link opens `/admin/affiliates/<id>`, directory filtering/pagination/page-size controls work, row details show commissions and safe account data, and the payout link goes to `/admin/payouts`. Mobile hides desktop sidebar, exposes menu links with usable targets, and does not create document overflow. **Permissions/negative:** Unauthorized/non-operator users receive the app’s access-denied/auth response and no directory/ledger. Customer affiliate data is not exposed to guests. Empty/error API responses show explicit states rather than stale fabricated rows.

AFF-06 - Admin payout queue transitions and payout detail drawer

Role/route: Authorized operator; `/admin/payouts`. **Preconditions:** Queue contains a requested payout. **Steps:**

1. Open from “View Payouts” and inspect payout amount, destination, and status.
2. Select “Approve”; then “Start processing”.
3. Select “Mark paid”, verify “Submit TXID” is disabled while TXID is blank, enter a transaction hash, and submit.
4. Select “View details”, inspect the “Payout details” drawer, then close it.
5. Repeat with empty and 503 queue responses and select “Retry” after error.

Expected: Valid transitions send `{ status: 'approved' }`, `{ status: 'processing' }`, then `{ status: 'paid', txid: '<hash>' }`; controls reflect each state. TXID is required for paid transition. Detail drawer shows amount, status, requested/approved/paid timestamps, destination wallet/network, and transaction hash with safe copy controls. Empty state says “No payout requests yet”; error state is distinct and Retry recovers when data returns. Mobile queue scrolls horizontally without document overflow. **Negative/edge:** Invalid transition, duplicate clicks, blank/short TXID, or API failure leaves status unchanged and reports an error. Only authorized operators can approve/reject/process/pay; customers can view only their own payout history and cannot call admin transitions.

AFF-07 - Admin affiliate program settings and provider controls

Role/route: Authorized owner/operator with settings permission; `/admin/affiliate-settings`. **Preconditions:** Existing settings load (commission, eligibility, payout minimum, cap, cookie duration, provider flags). **Steps:**

1. Inspect “Affiliate Program Settings”, “Program status”, commission rate, “Cookie duration”, “Minimum eligible”, “Payout Rules”, “USDT payout minimum”, and “Transaction cap”.
2. Open “Provider settings”; verify “Quickex integration” and “Manual adjustments”, then toggle one.
3. Change commission rate to `0.080`, optionally reset, then select “Save changes”.
4. Refresh and verify persisted settings; open “Documentation”/valuation guide.

Expected: Provider controls are collapsed initially, expand with `aria-expanded`, and hidden when collapsed. Save sends a new version (not an in-place destructive edit) with changed values and preserves unchanged settings; success refreshes settings. Reset restores loaded values and disables save when unchanged. Numeric fields enforce their declared ranges/steps; cookie duration is 1–365 days and transaction cap may be “None”.

Negative/permissions: Invalid/negative/over-limit rates, payout minimum, cap, or cookie duration are rejected client/server-side with no partial save. Unauthorized operators cannot load/save. A save error leaves edited values visible for correction, without claiming success.

AFF-08 - Admin valuation review queue approve/reject

Role/route: Authorized operator; `/admin/affiliate-settings` valuation section. **Preconditions:** A pending “Valuation Review Queue” entry with reason such as “Provider value unavailable.” **Steps:**

1. Inspect the queue and search field (`type=search`), clear search, choose filter/sort, refresh, select rows, and paginate.
2. Select “Approve”; enter a USD value in the confirmation dialog and submit.
3. Select “Reject” on another pending entry; enter a required explanatory note and submit.
4. Test blank approval value and blank rejection note; close with “Cancel”.

Expected: Queue supports search/filter/sort/refresh and mobile horizontal scrolling. Approve sends `{ state: 'approved', usd: '<exact value>' }`; reject sends `{ state: 'rejected', note: '<note>' }`. Dialog title changes between “Approve valuation” and “Reject valuation”; required fields block submission until valid. Confirmed rows leave/update pending state; cancellation sends nothing. **Negative/edge:** Non-numeric/negative/blank USD and blank notes are rejected; API failure retains pending row and shows an error. The queue does not reveal unrelated customer identity fields.

TG-01 - Telegram connect requires token and signed-in session

Role/route: Guest, signed-in customer; `/telegram/connect?token=<one-time-token>[&intent=signup]`.

Preconditions: Token may be valid, absent, expired, consumed, conflicting, or API unavailable. **Steps:**

1. Open without `token`; inspect the Connect Telegram card.
2. Open with a token while signed out; observe redirect to `/sign-in?redirect_url=...` (or `/sign-up` for intent `signup`, `sign-up`, or `register`).
3. Complete authentication and observe “Connecting Telegram...”.
4. With a valid token, inspect success; with invalid/expired/used/conflicting/401/403/5xx responses, inspect the error.
5. Reload/back-navigate and inspect the address bar.

Expected: Card heading is “Connect Telegram”. Missing token shows “Connection link unavailable” and asks for a new link. Signed-out token flow preserves the exact safe connect URL through auth. Signed-in flow POSTs once to `/api/telegram/connect` with `{ token }`, same-origin credentials, then shows “Telegram connected” and “Your QuickXchange account is now connected to Telegram...” on success. Error messages distinguish: already connected to another account (409), expired/already used (404/410/invalid/used/consumed), unverified session (401/403), or generic retry. Token is removed from browser URL/history after processing and never displayed. **Negative/security:** One-time consumption is guarded against duplicate effects; navigating away cancels UI updates. Do not retry by

resubmitting a token from the address bar; request a new Telegram link. A Telegram link cannot attach to a different authenticated account, and customer order notifications remain email-only (TG-01 is account linking, not an order notification channel).

Admin operations and reporting

Inventory (routes, implementation, tests)

- `/admin` -> `AdminOverview` (`artifacts/crypto-exchange-widget/src/pages/admin.tsx`), chart components in `artifacts/crypto-exchange-widget/src/pages/admin-overview-charts.tsx`; route registration `artifacts/crypto-exchange-widget/src/App.tsx`.
- `/admin/orders`, `/admin/orders/:id` -> `AdminOrders` and `OrderDrawer` (`admin.tsx:4746-5150`, `drawer/detail` roughly `admin.tsx:5152-6188`); `e2e/admin-orders.spec.ts` is the primary route/detail/status/archive/export test.
- `/admin/customers`, `/admin/customers/:id` -> `AdminCustomers` (`admin.tsx:6189-6270`) and `AdminCustomerProfile` (`artifacts/crypto-exchange-widget/src/pages/admin-customer-profile.tsx`); `e2e/admin-customer.spec.ts`.
- `/admin/revenue` -> `AdminRevenue` (`admin.tsx:38904-40444`); no dedicated E2E found, but revenue is covered by `e2e/admin-overview.spec.ts` fixtures and `artifacts/api-server/test/manual-desk-revenue.test.ts`.
- `/admin/notification-settings` -> `AdminNotificationSettings` (`admin.tsx:1837-2455`); no dedicated E2E found; notification API route is `artifacts/api-server/src/routes/notification-settings.ts`.
- Role distinction: all pages enforce `AdminShell` permissions: `overview/revenue` require `statistics.view`; `orders` require `orders.view`; `customer list/profile` require `customers.view`; `notification settings` requires `site_settings.manage` and additionally renders `Owner access required` unless `isOwner`. Order mutations are permission-gated (`orders.status`, `orders.confirm_payment`, `orders.complete`, `orders.cancel`, `orders.archive`, `orders.export`); `owner-only overview diagnostics/config sync` and `Telegram health` are explicit. `Swap/manual orders` are operator-editable through permitted later settlement states; `Convert/instant statuses` are provider-synchronized and `bulk status` is disabled.

ADM-OV-01 - Swap overview loads scoped analytics and operational surfaces

Role/route: Admin/operator with `statistics.view`; `/admin`. Preconditions: Summary API returns Swap data with total/pending/completed/failed counts, daily series, rankings, recent activity, and manual orders. Steps:

1. Navigate to `/admin`.
2. Observe product control `Swap` selected and default range `7D`.
3. Inspect KPI cards, `Orders Over Time`, `Exchange Volume`, `Top Trading Pairs`, `Orders Overview`, `Top Currencies`, `Live Exchange Activity`, `Needs Attention (Swap)` and `Recent Orders (Swap)`.
4. Select `View full queue`, `View all orders`, or a row's `View order` link.

Expected (observable): Summary and active/recent order requests include `product=swap`, `type=manual`, UTC start at `T00:00:00.000Z` and end at `T23:59:59.999Z`; eight metric cards render; chart labels and status distribution match API; queue/recent rows show exchange, send/receive, status, paid state, user, timestamp and `View order`; links preserve the selected date/type scope. Empty lists show `The queue is clear—nice work/No recent orders found`.

ADM-OV-02 - Convert overview switches product and exposes provider warning

Role/route: Admin/operator with `statistics.view`; `/admin`. Preconditions: Convert summary has `pendingOrders`, `dailySeries`, provider health with non-healthy Quickex reconciliation. Steps:

1. Click `Convert` (`button-overview-convert`).
2. Wait for summary and queue refresh.
3. Inspect `Total Profit`, operational warning, queue and charts.

Expected (observable): Requests use `product=convert`, queue uses `type=instant`; Convert activity appears; profit reads Not tracked with Unavailable for Convert; Swap-only Manual Swap Blockchain Monitoring is absent while Blockchain Watches remains; warning says Quickex reconciliation is delayed after the reported consecutive failures and that Convert status/affiliate credits may be delayed. Negative: a healthy reconciliation response produces no warning.

ADM-OV-03 - Preset/custom inclusive UTC date boundaries

Role/route: Admin/operator; `/admin`. Preconditions: Summary/order mocks record request URLs. Steps:

1. Click `24H`, `7D`, `30D`, and `90D` in turn.
2. Click `Custom`.
3. Fill `Overview start date` with `2025-03-01` and `Overview end date` with `2025-03-03`.
4. Inspect the range label and request query values.

Expected (observable): Presets use UTC midnight through UTC `23:59:59.999`; custom requests use `from=2025-03-01T00:00:00.000Z` and `to=2025-03-03T23:59:59.999Z`, and queue `createdFrom/createdTo` exactly match. Date inputs enforce `max/min`, preventing end-before-start. Edge: incomplete custom dates retain the prior bounds until both exist.

ADM-OV-04 - Partial/unavailable valuation and unaudited revenue states

Role/route: Admin/operator; `/admin`. Preconditions: Summary valuation is `partial` or `unavailable`; revenue has no normalized completed total or request errors. Steps:

1. Load overview with valuation missing BTC rates.
2. Read `Total Volume` and `Average Order Value` notices.
3. Return a revenue response with no completed audited total; repeat with revenue request failure.

Expected (observable): Partial state shows Partial USD Valuation and Missing rates for BTC. Only X of Y valued; unavailable state says USD valuation is unavailable/cannot compute approximate USD. Profit shows Unavailable with No audited completed revenue, or Audited revenue report could not load; no fabricated zero profit is shown. Chart with empty `dailySeries` renders No data available.

ADM-OV-05 - Queue search/status filtering and empty outcome

Role/route: Admin/operator; `/admin`. Preconditions: Queue has multiple active rows with differing IDs/customers/statuses. Steps:

1. In `Needs Attention`, fill `Search queue` with an order ID, asset, customer, or status fragment.
2. Click `Filter` and choose `Awaiting`, `In progress`, or `Confirmed` from the status selector.
3. Enter a term that matches nothing.

Expected (observable): Filtering is case-insensitive/trimmed across order ID, assets, customer name/email and rendered status; table updates without a server request; no-match state says No queue items match your search and Try another search or filter. Edge: empty source queue instead says The queue is clear—nice work.

ADM-ORD-01 - Orders directory switches Swap/Convert and archive views

Role/route: Admin/operator with `orders.view`; `/admin/orders`. Preconditions: Swap/manual and Convert/instant fixtures plus archived fixtures. Steps:

1. Navigate to `/admin/orders`.
2. Verify tabs `Swap` and `Convert`, then click `Convert`.
3. Click `Archived Orders`, then `Active Orders`.

Expected (observable): Swap uses manual settlement options/status labels; Convert uses instant options and provider statuses. Changing product clears send/receive method and status filters. Archived request has archived=archived; Active has archived=active; table columns include Order ID, Exchange, Send, Receive, Status, User, Created At. Anonymous rows say Anonymous plus email; registered rows say Registered user.

ADM-ORD-02 - Orders search, advanced filters, sort, date and amount boundaries

Role/route: Admin/operator; /admin/orders. Preconditions: API records /api/orders query parameters. Steps:

1. Click **Filters**.
2. Set **Status**, **Send method**, **Receive method**, **User email**, **Rate mode** (Floating/Fixed), **Outcome** (Unknown / held or Known), **Min amount**, **Max amount**, **Provider state**, **Newest first/Oldest first**, **Created from**, and **Created to**.
3. Fill the main search with surrounding whitespace and wait for debounce.
4. Click the clear-filters X (**Clear filters**).

Expected (observable): Query contains normalized settlement IDs, email/search, status, outcome, numeric amounts, provider state, sort and inclusive UTC dates; search/customer email debounce before request; active-filter count appears beside **Filters**; **Clear filters** resets all filters and sort to newest. Negative: invalid/no matching combinations show No swap orders match these filters /corresponding Convert empty state, not stale rows.

ADM-ORD-03 - Export orders XML and permission/error handling

Role/route: Admin/operator with orders.export; /admin/orders. Preconditions: export endpoint returns XML, then simulate failure. Steps:

1. Click **Export XML**.
2. Wait for browser download.
3. Simulate export failure and click the inline **Try again**.

Expected (observable): Button changes to **Preparing**; download filename is orders-YYYY-MM-DD.xml; object URL is cleaned up. Failure displays **Export failed**, The XML export could not be prepared. Try again in a moment. and retry. Customer-scoped orders (/admin/orders?customerId=...) intentionally have no export button. Role edge: operator without export permission cannot see it.

ADM-ORD-04 - Bulk Swap status transition with shared legal status

Role/route: Admin/operator with orders.status and specific confirm/complete/cancel permissions; /admin/orders Swap Active. Preconditions: select two manual orders sharing a later legal transition. Steps:

1. Check each row or **Select all displayed Swap orders**.
2. Verify **N orders selected** and click **Change status**.
3. Select **Completed** in **New status** and click **Change status**.

Expected (observable): Confirmation says the status applies to selected Swap orders and preserves audit/notification history; payload includes each {id, recordVersion} plus manualSettlementState=completed; success notice says N updated successfully; selection clears and data refreshes. Edge: button is disabled with The selected Swap orders do not share a later status when no common legal transition; Convert button disabled with provider-synchronization explanation.

ADM-ORD-05 - Archive conflict gives partial result without silent loss

Role/route: Admin/operator with orders.archive; /admin/orders Swap Active. Preconditions: two selected rows, one succeeds and one returns ORDER_VERSION_CONFLICT. Steps:

1. Select both rows.
2. Click **Archive Selected**.

3. In confirmation, verify text `Nothing is permanently deleted`, then click `archive`.

Expected (observable): Payload is `archived: true` with current record versions; result notice says `1 archived; 1 failed` and includes conflict summary; successful row moves out/refreshes while failed row remains actionable. `Escape` or `Cancel` closes dialog without a request and restores focus.

ADM-ORD-06 - Archived restore and permanent delete safeguards

Role/route: Admin/owner with `orders.archive`; `/admin/orders` -> `Archived Orders`. Preconditions: archived orders exist. Steps:

1. Select archived rows and click `Restore Selected`.
2. Confirm dialog text says they return to `Active Orders`; observe `2 restored successfully`.
3. Select archived rows again and click `Delete Selected`.
4. Verify `Permanently delete N archived orders?` and `This action cannot be undone.`; cancel once, reopen and click `Delete Permanently`.

Expected (observable): Restore sends `archived:false`; delete is only available in Archived view, requires explicit destructive confirmation, sends IDs/record versions, removes successful rows immediately, and shows empty state/count update. Negative: active orders never expose permanent delete; cancelled/escaped dialogs do not mutate.

ADM-ORD-07 - Order detail drawer exposes operational data but hides internal metadata

Role/route: Admin/operator; `/admin/orders/:id?type=instant` or `?type=manual`. Preconditions: detail contains addresses, transaction hash, `step2Details`, provider/funding internal metadata and assignment. Steps:

1. Open a row (URL retains query context).
2. Inspect `Order status progression`, `Admin Order Information`, `Exchange Details`, `Additional Payment Details`, and `Transaction Details`.
3. Use copy icons/tooltips `Copy Order ID` and `Copy Sending Address`; click `Copy Info` and `Close`.

Expected (observable): Detail shows customer send/receive, status, assigned operator, relevant `Contact Email`, `Customer IBAN`, `Destination Memo / Tag`, and transaction hash in the transaction section; provider-only fields (network code, source, internal status, deposit provider, selected provider, logo URL, Quote ID) do not leak into customer/additional details. `Copy Info` changes to `Copied ✓`; `Close` returns to list. Convert provider-synchronized rows cannot be admin-status-edited.

ADM-ORD-08 - Detail invoice availability and customer-facing projection integrity

Role/route: Admin/operator; `/admin/orders/:id`. Preconditions: completed manual and Convert orders with tracking capability and projection endpoints. Steps:

1. Open completed manual order and click `Download Invoice`.
2. Verify download is a one-page A4 PDF with route, amounts, fee and customer step-2 fields.
3. Open completed Convert order and download its invoice.

Expected (observable): filename is `quickxchange-invoice-{id}.pdf`; manual invoice includes `Exchange USDT to EUR`, `10 USDT`, `8.33 EUR`, `IBAN/account holder/memo` and fee; internal provider reference, provider fields and transaction ID/hash are absent. Convert invoice includes exchange/amounts but omits `Fee`, provider-only details and payment/receiving information. Pending/in-progress orders have no admin invoice button. Status refresh requests use the order tracking token.

ADM-CUS-01 - Customer directory search and responsive navigation

Role/route: Admin/operator with `customers.view`; `/admin/customers`. Preconditions: customer list includes named and unnamed users. Steps:

1. Fill `Search by name or email` with a name, then email, then an unmatched value.
2. Click a desktop row or mobile customer card.
3. Repeat at 1280, 960, 768, 390 and 320px widths.

Expected (observable): Search is debounced and list updates; rows show Customer, Orders/lifetime, Volume/settled, Last activity and Status; unmatched state says `No customers found`; desktop uses `table-customers`, `<=820px` uses `users-mobile-cards/user-card-*`; no horizontal page overflow; click/keyboard Enter/Space navigates to `/admin/customers/:id`. Footer says `Showing N records`.

ADM-CUS-02 - Customer profile details, roles and stats-to-orders scope

Role/route: Admin/operator with `customers.view`; `/admin/customers/CUST-123`. Preconditions: customer has verified email, affiliate code/rate, order/sales/referral stats and send/receive volumes. Steps:

1. Open profile and inspect header badges and `User Details`.
2. Verify `Personal Information`, `Affiliate Account`, role and account status.
3. Click `Stats`; inspect `Total Orders`, `Done Orders`, `Balance Owed`, `Total Sales`, `Referred Users`, `Total Referral Profit`, `Send Volume`, `Receive Volume`.
4. Click `Total Orders`.

Expected (observable): Header shows name/email, `Confirmed` or `Unconfirmed`, and `Active/Suspended`; roles render `Customer/Affiliate/VIP`; fiat amounts use USD formatting and crypto assets retain symbols. `Total Orders` navigates to `/admin/orders?customerId=CUST-123`, requests immutable `customerId` and `archived=all`, selects `All Orders`, shows customer scope, and intentionally hides XML export.

ADM-CUS-03 - Customer referrals pagination and empty/error handling

Role/route: Admin/operator; `/admin/customers/:id` -> `Referred Users`. Preconditions: referrals endpoint returns items, then zero, then error. Steps:

1. Click `Referred Users`.
2. Inspect Name, Country, Number of Orders, Registered At and pagination text.
3. Click `Next`, then `Previous`; repeat with no referrals and an API error.

Expected (observable): Page size is 25; `Previous` is disabled on page 1, `Next` disabled at total; rows render dates/counts, horizontal table scrolling does not overflow page. Empty state says `No referred users with description`; error shows `referralsLoadError` and `retry`.

ADM-CUS-04 - Edit profile validation, role/referral controls and save outcome

Role/route: Admin/operator with customer update permission; `/admin/customers/:id`. Preconditions: profile loaded. Steps:

1. Click `Edit Profile`.
2. Edit `First Name`, `Last Name`, two-letter `Country`, `Role` (`Customer/Affiliate/VIP`), `Custom Referral Code`, and `Referral Rate (%)`.
3. Click `Save Changes`; then simulate API failure and `retry`.

Expected (observable): Country uppercases and maxes at 2 chars; referral code uppercases, max 12 and pattern-constrained; rate accepts 0-100 with 0.01 step; payload trims names/country, converts percentage to decimal, and only sends changed code. Success closes dialog, updates profile/list caches and shows success notice; failure keeps dialog/error notice and does not claim success. `Cancel/close` discards edits; `Save` is disabled in flight.

ADM-CUS-05 - Password reset confirmation and negative paths

Role/route: Admin/operator with customer-management permission; `/admin/customers/:id`. Preconditions: profile loaded. Steps:

1. Click `Reset Password`.
2. Enter mismatching values and submit.
3. Enter a matching password under 12 characters and submit.
4. Enter matching 12+ character values and click `Continue`; review confirmation containing customer email; click `Set Password`.
5. Cancel or `Back` at each dialog stage.

Expected (observable): Errors are `Passwords do not match` and `minimum-length` message; no reset request occurs. First valid submit advances to confirmation only; second submits `temporaryPassword`, closes/resets form, and shows success. `Back` returns to fields; `Cancel` clears values; controls remain in-viewport at small widths.

ADM-CUS-06 - Email confirmation, suspend/activate and revoke-session safety

Role/route: Admin/operator with customer permissions; `/admin/customers/:id`. Preconditions: unverified active customer; separately suspended customer; session-revocation API can return unsupported. Steps:

1. For unverified user click `Mark Email Confirmed`.
2. For active user click `Suspend User`, read confirmation, cancel, reopen and confirm.
3. For suspended user click `Activate User`.
4. Click `Revoke All Sessions`, cancel once, reopen and confirm.

Expected (observable): `Confirm` button disappears and badge changes to `Confirmed` only after refetch reports `emailVerified`; failed/proof-not-reflected response shows error. `Suspend` confirmation is explicit; success changes badge to `Suspended` and action to `Activate User`; `activate` reverses it. `Revoke` confirmation includes email; success notice confirms all sessions revoked; `CLERK_SESSION_REVOCATION_UNSUPPORTED` shows specialized unsupported message. No `cancel` action mutates; buttons disable during requests.

ADM-REV-01 - Revenue report filters and route/pricing-rule grouping

Role/route: Admin/operator with `statistics.view`; `/admin/revenue`. Preconditions: manual-desk report has totals, normalized totals and groups. Steps:

1. Set `From`, `To`, and `Reporting Currency` (input uppercases); verify default 30-day range.
2. Select `Group results by = Source / target route`, inspect `Revenue Breakdown`.
3. Select `Pricing rule version`.

Expected (observable): Query uses inclusive UTC bounds, `reportingCurrency` and `groupBy=route/pricingRule`; status cards show `active/completed/failed/cancelled` expected fees and gross volume per target asset; table columns are grouping label/key, `Status`, `Orders`, `Gross Customer Volume`, `Expected Fee Revenue`, `Historical Normalized Fee`. Invalid date order is constrained by input min/max; API error gives retryable `Load revenue report error`; no rows gives `No snapshotted manual orders in this period`.

ADM-REV-02 - Exact decimals and CSV export

Role/route: Admin/operator; `/admin/revenue`. Preconditions: report contains high-precision decimal strings and CSV endpoint can succeed/fail. Steps:

1. Inspect rounded fiat/crypto values and hover a value for exact title.
2. Click `View exact values`.
3. Click `Export CSV`; then simulate failure and use `retry` notice.

Expected (observable): Toggle changes to Exact decimals on and renders full source decimal strings without floating-point corruption; title reads Exact value: ...; report identifies snapshot rates/normalization decimal scale. CSV button changes to Preparing, downloads manual-desk-revenue-YYYY-MM-DD-YYYY-MM-DD.csv; failure says The revenue CSV could not be prepared. Try again in a moment. and no misleading download occurs.

ADM-NOT-01 - Owner-only notification settings and event matrix

Role/route: Owner with `site_settings.manage`; `/admin/notification-settings`. Preconditions: settings API returns draft flags. Steps:

1. Navigate to route as Owner.
2. Inspect Admin Notifications, master status, event rows and Email/Telegram switches.
3. Toggle Notifications are enabled, New Order (Created), Payment Received, Order Processing, Order Completed, and Order Failed / Cancelled channels.
4. Click SAVE NOTIFICATION SETTINGS.

Expected (observable): Draft changes are local until save; payload includes admin/customer event flags, channel enables, email/phone/chat IDs and Trustpilot URL; success says Settings saved successfully. and refreshes draft. A non-owner sees Owner access required rather than controls; loading shows LoadingBlock.

ADM-NOT-02 - Admin email prerequisites/test and customer template editor

Role/route: Owner; `/admin/notification-settings`. Preconditions: saved admin email, template list; also test unsaved/invalid states. Steps:

1. Edit Admin notification email; observe Save this address before sending a test email and disabled Send Test Email.
2. Save, then click Send Test Email; inspect success/failure result.
3. Toggle Master Switch under Customer Email Notifications (Examples).
4. Toggle each example: Order Created, Payment Received, Processing, Completed / Done, Failed / Cancelled.
5. Click Edit Templates, choose event kind, edit subject/heading/message/button/footer, insert/use variables such as `{{customerName}}`, `{{orderId}}`, `{{status}}`, `{{orderUrl}}`, `{{invoiceUrl}}`, `{{trustpilotUrl}}`, then save/test.

Expected (observable): Test email is enabled only when persisted email equals draft; result displays returned message. Disabled examples are visibly faded/grayscale; master/event switches persist in the settings save payload. Template save says Templates saved successfully.; test sends the selected draft and reports returned success/error. Negative: missing template is a no-op; API errors show failure without losing draft.

ADM-NOT-03 - Telegram health, connect/link expiry, test and disconnect

Role/route: Owner; `/admin/notification-settings`. Preconditions: Telegram health may be healthy, webhook-mismatched, bot-invalid or unavailable. Steps:

1. Inspect Telegram Notifications, Telegram Phone Number, read-only masked Telegram Chat ID, and health status.
2. With no link click Connect Telegram; complete the bot flow, then observe pending polling.
3. Simulate connected, expired, bot verification failure and webhook mismatch.
4. When connected click Send Test Telegram, then Disconnect.

Expected (observable): Health renders verified bot/webhook/error diagnostics; Connect is disabled for bot identity error or configured=false; no-link state says No Admin Telegram chat is linked. Telegram alerts cannot be sent until connection succeeds. Pending link polls every 3s; connected state says Telegram successfully connected, masks chat ID and enables Telegram master toggle/test/disconnect; expired says Telegram connection

link expired. Test shows returned result; disconnect clears chat/settings and says Telegram disconnected. Phone number is displayed as contact-only and never treated as Chat ID.

Admin catalog, pricing and provider controls

Inventory

Routes/tabs

- `/admin/currencies` (`requiredPermission=catalog.view`): **Currencies, Payment Methods, Crypto Assets, Crypto Networks**. Catalog search is shared; each tab has create/edit/delete, selection checkboxes and contextual bulk actions. The old standalone customer-Deposits panel is absent. Customer deposit configuration is now per exact crypto **Asset + Network**, in the network edit drawer, section **5 / Visibility — Customer Deposits / Widget** (`admin.tsx:10635-10895`).
- `/admin/pricing` (`pricing.view`): manual Swap pricing rules; path pricing and amount/range pricing modes; individual test/preview, add/edit/delete, selection and bulk edit.
- `/admin/providers` (`providers.view`): provider cards for **Quickex** and **WhiteBIT**, health/status summaries and refresh; Quickex credentials/configuration and diagnostics; WhiteBIT credential/status, mapping and exact-route address-permission verification.
- `/admin/integrations` (`integrations.view`): API Integrations, currently Quickex integration workflow. Provider links jump to `#integration-quickex` and `#integration-whitebit`.
- Blockchain Monitoring is exposed from a crypto-network edit drawer as **Blockchain Monitor**, with network configuration, asset identity and readiness; server routes additionally expose setup routes, watches, matches, registration gaps and ready-route enable operations (`admin-blockchain-monitor-config.tsx:17-367`; `routes/blockchain-monitoring.ts:34-456`).
- Admin overview has owner-only **Workspace configuration sync** (`admin.tsx:3733-3770`): import JSON, preview, inspect counts, apply reviewed changes.

Actions/fields/toggles

- Fiat currency: add/edit `Code`, `Name`, `Precision`, `Network`, lifecycle/regions/countries and enabled state; search, edit, delete. Payment method: `ID`, `Name`, description/instructions, logo upload/replace, `Enabled`, `Can send`, `Can receive`, field-definition menus (including receive email), attach/detach from currencies, direction/instruction/min/max overrides. Crypto asset: code/name/decimals/logo/enabled, edit/delete and bulk enabled/disabled/delete. Crypto network: network name/code, decimals, enabled, confirmations/guidance, explorer URL, deposit instructions/warning, memo requirement, shared address/memo, provider/fallback and customer-deposit controls.
- Network bulk utilities: **Assign Deposit Provider**, providers **WhiteBIT, Manual Wallet, None; Select All Supported, Clear All**, route search, exact-route checkboxes, WhiteBIT mapping select, **Review Changes, Back, Confirm Assignment**. It explicitly does not create a WhiteBIT address and does not change saved manual addresses/memos/manual wallet tracking (`bulk-deposit-provider-dialog.tsx:195-307`).
- WhiteBIT asset sync dialog: **Sync with WhiteBIT**, preview totals `WhiteBIT assets`, `Already in QuickXchange`, `New assets`; search missing assets, per-page `15/25/50/100`, select-all-visible, individual selection, `Previous/Next`, **Import Selected**, success summary/Close, retry on preview failure (`admin-whitebit-asset-sync-dialog.tsx:118-330`).
- Pricing rule path fields: name, source/target settlement-option selectors, exact rate, minimum/maximum quantity, expected settlement time, priority, operator/customer instructions, enabled, markup/percentage/fixed fee/direction. Path pricing and Amount/range pricing are exclusive editors. Range table columns: `Min`, `Max`, `Fixed`, `Percentage`, `Direction`, `Actions`; ranges support no-limit max, `MARKUP`, `GIVE_MORE`; add/edit/delete. Bulk fields are opt-in checkboxes so unchecked fields are omitted.

- Blockchain Monitor fields: `Adapter` = EVM/Solana/Tron/Bitcoin/TON/Stellar; `Provider` = RPC Node/Indexer/Alchemy/Helius; `Endpoint Secret Name` (Optional), `API Key Secret Name` (Optional), `Chain ID` (Optional), `Poll Interval` (Seconds), `Confirmations Required`, `Finality Policy` = Confirmations/Finalized/Instant/Strict/Optimistic, **Enable Network Monitor**, **Create/Update Network Config**, **Test Connection**. Asset: `Identity Kind` = Native Asset/Token, `Contract / Mint Address` (Tokens only), `Decimals`, **Enable Asset Monitoring**, **Save Asset Identity** (`admin-blockchain-monitor-config.tsx:253-367`). Status is Network Monitor, Asset Monitoring, Provider, Last Scan, Last Error and readiness `READY/BLOCKED` with reasons.
- Config sync preview counts categories: crypto assets/networks, fiat currencies, payment methods, fiat-method links, manual pricing rules, customer-deposit eligibility, site pages/publication and landing background. Actions are file select, preview and **Apply reviewed changes**; apply requires confirmation `WORKSPACE_CONFIG_APPLY` and the preview state hash (`workspace-config-sync.ts:321-341`).

Scenarios

C01 - Inventory catalog tabs and removed Deposits panel

Role/route: Catalog operator, `/admin/currencies`. **Preconditions:** Catalog permission; mocked/API catalog returns fiat, method, crypto asset and network rows. **Steps:** 1. Open Currencies. 2. Click **Payment Methods**, **Crypto Assets**, and **Crypto Networks**. 3. Search each tab with a mixed-case term. 4. Inspect the page and network edit drawer for a standalone `Deposits` tab/panel. **Expected (observable):** Four tab labels are available; search filters matching rows and clear restores them. No standalone customer-Deposits panel appears (`e2e/admin-currencies.spec.ts`); deposit settings occur per network under **Customer Deposits / Widget**.

C02 - Add/edit/delete fiat currency

Role/route: Catalog manager, `/admin/currencies?tab=currencies`. **Preconditions:** `catalog.view` and `catalog-management` permission; USD/EUR fixture. **Steps:** 1. Click **Add currency**. 2. Enter `CAD` and `Canadian Dollar`; set `precision/network/region` fields as required. 3. Click **Save currency**. 4. Search `CAD`, edit it, change a value, save. 5. Select it and choose **Delete**, accepting confirmation. **Expected:** New row appears with entered code/name; edit persists; delete removes it. Blank/invalid code, duplicate code or invalid precision is rejected and no API write occurs. E2E confirms add/search/edit/delete flows (`e2e/admin-currencies.spec.ts`).

C03 - Create payment method with logo and dynamic field

Role/route: Catalog manager, `/admin/currencies?tab=methods`. **Preconditions:** Payment-method management permission; logo upload endpoint available. **Steps:** 1. Click **Add payment method**. 2. Fill ID `interac`, name `Interac e-Transfer`. 3. Upload a logo; verify button changes to **Replace**. 4. Set **Enabled**, **Can send**, **Can receive** as appropriate. 5. Use **Add receive field** then select **Email**. 6. Click **Save payment method**; edit and save again. **Expected:** Row and image appear; stored object path is retained on re-save; receive field is present. Missing ID/name, invalid image type/size or failed upload displays validation/error and does not save (`e2e/admin-currencies.spec.ts`).

C04 - Attach payment method and bulk direction/conflict behavior

Role/route: Catalog manager, `currencies` tab/edit drawer. **Preconditions:** Currency and reusable method exist; attachment permission. **Steps:** 1. Edit USD. 2. In attachment selector choose `Interac e-Transfer`. 3. Click **Attach**. 4. Set `send/receive/min/max/instructions` overrides if shown and save. 5. Attempt attaching the same method again; use bulk attach/update preview if available. **Expected:** Attachment appears with `enabled=true`; duplicate preview is `conflict ( PAYMENT_METHOD_ATTACHMENT_EXISTS )` and cannot silently duplicate. Direction resolves to `both`, `send`, `receive` or `none`; bulk preview must be reviewed before apply (`e2e/admin-currencies.spec.ts`; fixture preview lines 120-171).

C05 - Crypto asset/network bulk utilities

Role/route: Catalog manager, `Crypto Assets/Crypto Networks`. **Preconditions:** BTC, USDT/TRC20/ERC20 and XMR fixtures; bulk permission. **Steps:** 1. Select BTC in `Crypto Assets` and choose **Disabled**. 2. Select Bank transfer in `methods` and choose **Disabled**. 3. Select Ethereum network and click **Delete**, accept browser confirmation. 4. Select BTC asset and delete, then delete Bank transfer. **Expected:** Selected records change `enabled` state or disappear; delete is confirmation-protected. At mobile widths 390/768 there is no horizontal overflow and network rows remain usable (`e2e/admin-currencies.spec.ts`).

C06 - Bulk assign deposit provider with exact mappings

Role/route: Catalog manager, Crypto Networks. **Preconditions:** Selected network routes; WhiteBIT catalog/status endpoints available. **Steps:** 1. Select routes and click **Assign Deposit Provider**. 2. Verify **Set Deposit Provider** defaults to WhiteBIT. 3. Search **XMR**; select exact route checkboxes. 4. Use **Select All Supported** and confirm unsupported BTC is not selected. 5. If multiple options exist, choose the exact **WhiteBIT Network for [asset] · [network]**; do not rely on guessing. 6. Click **Review Changes**, inspect route/mapping/customer-deposit lines, then **Confirm Assignment**. **Expected:** Readiness shows route support, mapping, credentials, address permission and Ready for Widget; unresolved/unsupported mappings disable apply. Review states exact routes and Customer Deposits: ...; successful apply updates provider and invalidates catalog/config/status (bulk-deposit-provider-dialog.tsx:183-181,288-307; e2e/admin-currencies.spec.ts).

C07 - Prevent provider switch while Customer Deposits are enabled

Role/route: Catalog manager, network drawer/bulk dialog. **Preconditions:** Exact route has Customer Deposits ON. **Steps:** 1. Open route edit and observe persisted **Customer Deposits** state. 2. Attempt switching provider or bulk assigning a different provider. 3. In the separate section click **Turn Customer Deposits OFF** and save. 4. Return to provider assignment, review again, apply. **Expected:** First attempt is blocked with Turn off Customer Deposits ... in their own controls; manual addresses/memos remain unchanged. After explicit OFF and a fresh review, provider switch can proceed; it does not create a WhiteBIT address (bulk-deposit-provider-dialog.tsx:145-151,177-181; admin.tsx:10310-10323).

C08 - WhiteBIT asset sync preview/import

Role/route: Catalog manager, Crypto Assets, **Sync with WhiteBIT**. **Preconditions:** WhiteBIT catalog has missing candidates. **Steps:** 1. Open sync dialog. 2. Verify preview totals. 3. Search ticker/name/network; change per-page 15→25. 4. Select individual assets and/or select all visible. 5. Navigate **Next** and back; verify selection reset behavior. 6. Click **Import Selected**, then **Close**. **Expected:** Only missing candidates are listed; network badges show deposit/withdraw/memo/confirmations. Import success reports imported/skipped counts and refreshes assets/networks. Empty search says No options; preview failure offers **Retry** (admin-whitebit-asset-sync-dialog.tsx:127-170,174-220,285-330).

C09 - Create a pricing rule; edit path quantity limits

Role/route: Pricing manager, /admin/pricing. **Preconditions:** pricing.view/pricing.manage; settlement options include USD bank and EUR bank. **Steps:** 1. Click **Add pricing rule**. 2. For a single directed source/target, **Adding Range** is the initial editor. Fill name, source, target, exact path rate and a valid amount range, then save. 3. Reopen the saved rule and select **Edit Path**. 4. Set optional Minimum Quantity and Maximum Quantity; try 0, -1, 1e4, excessive precision and maximum below minimum. 5. Correct the limits and save. **Expected:** Invalid input shows positive decimal; reversed bounds show must not exceed; no request is sent. Valid save closes drawer and creates rule (e2e/admin-pricing.spec.ts).

C10 - Range mode, no-limit and overlap validation

Role/route: Pricing manager, pricing rule drawer. **Preconditions:** Existing rule; mobile viewport optional. **Steps:** 1. Open rule and select **Adding Range**. 2. Click **Add Range**. 3. Fill Min, Max, Percentage, Fixed Fee and choose **MARKUP**; alternatively check no-limit max. 4. Try blank Fixed Fee and overlapping 700–900 over existing 800–20000. 5. Correct to disjoint ranges; edit direction to **GIVE_MORE**; delete and recreate a range; save. **Expected:** Range table columns are Min, Max, Fixed, Percentage, Direction, Actions; missing Fixed Fee is rejected, overlap displays overlap, No Limit is visible, and Give more (customer bonus) is shown after edit. Path quantity fields appear only in **Edit Path**; adding or editing ranges does not erase saved path values (e2e/admin-pricing.spec.ts).

C11 - Pricing preview uses matched rule and neutral unknown fees

Role/route: Pricing manager, pricing rule test/preview. **Preconditions:** Competing rules and quote-preview endpoint. **Steps:** 1. Click a rule's test action. 2. Click **Preview pricing** repeatedly for a matched range, unknown rule and base fallback. 3. Observe rule name, percentage and commission panel. **Expected:** Matched range shows correct rule and +2.375%/Customer bonus; unknown rule says See quote fee and displays actual percentage fee rather than 0%; fallback shows signed commission such as -0.75% (e2e/admin-pricing.spec.ts).

C12 - Bulk pricing edit is opt-in and handles partial results

Role/route: Pricing manager, /admin/pricing. **Preconditions:** Multiple visible rules, one possibly read-only/legacy ambiguous. **Steps:** 1. Select visible rules (or **Select All** filtered rules). 2. Click **Bulk edit**. 3. Check **Apply markup**,

enter 2.50, leave amount fields unchecked; save. 4. Reopen, check amount-based pricing, add two ranges, save. 5. Reopen and clear/disable amount pricing; save. **Expected:** Unchecked fields are omitted and existing ranges are not reactivated; checked tiers apply to every selected rule. Read-only/conflicting rows are skipped and the result reports affected/updated/skipped diagnostics (`e2e/admin-pricing.spec.ts`).

C13 - Quickex diagnostics and credential secrecy

Role/route: Provider/integration owner, `/admin/integrations` or `/admin/providers`. **Preconditions:** Owner; Quickex status endpoint reachable. **Steps:** 1. Click **Test signed Order API**/diagnostic. 2. Click **Add Integration** or **Configure Quickex**. 3. Enter PIN code (4-8 digits), click **Continue**, then enter Public API Key and Secret API Key. 4. Click **Integrate Quickex**. 5. Reload and open Configure again. **Expected:** PIN/key inputs are password type; keys disabled until PIN continuation. Success says `Quickex signed Order API connected`, fields clear and remain blank/disabled after reload; UI shows `Encrypted storage` and never renders key values. Diagnostic reports authenticated/reachable (`e2e/admin-integrations.spec.ts`; `admin.tsx:1126-1166`).

C14 - Provider status unknown, Quickex health and WhiteBIT separation

Role/route: Provider operator, `/admin/providers`. **Preconditions:** Test one run with status 503; second with stored signing keys and reconciliation failures. **Steps:** 1. Load Providers while Quickex status fails. 2. Observe summary cards. 3. Load healthy stored-key state. 4. Inspect Legacy API Key, reconciliation warning, API availability/authentication/market data/order placement/policy. 5. Inspect WhiteBIT card and its **Address creation permission**. **Expected:** Failed status leaves summary values as `—`, not false zeros. Missing legacy API key is **Not required** when signing keys are active; two consecutive failures warn that affiliate completion/reversal credits may be delayed. WhiteBIT permission is independently `Verified/Not verified`; credentials alone do not enable address creation (`e2e/admin-integrations.spec.ts`; `admin.tsx:1364-1478,1526-1555`).

C15 - WhiteBIT exact-route permission verification

Role/route: Owner, `/admin/providers` WhiteBIT card/network route drawer. **Preconditions:** WhiteBIT credentials stored; exact Asset + Network mapping selected; current verification catalog. **Steps:** 1. Select exact asset/network. 2. Run signed credential check. 3. Acknowledge permission confirmation. 4. Confirm verification-address request if prompted. 5. Recheck status and then separately enable Customer Deposits only when readiness is current. **Expected:** UI states that a real verification address may be created/discarded; current proof can avoid another creation. Verification does not change provider settings and Customer Deposits stay OFF until separately enabled. Missing credentials/mapping/proof blocks enablement with an actionable error (`admin.tsx:10229-10263,10635-10689`).

C16 - Configure blockchain network and asset identity

Role/route: Monitoring operator, network edit drawer → **Blockchain Monitor**. **Preconditions:** Crypto network exists; secret names may be provisioned in environment. **Steps:** 1. Choose Adapter and Provider. 2. Enter endpoint/API-key secret names, optional Chain ID, poll interval and confirmations. 3. Choose Finality Policy; toggle **Enable Network Monitor**. 4. Click **Create/Update Network Config**. 5. Click **Test Connection**. 6. Choose Native Asset and save, then switch to Token and enter Contract/Mint, decimals and **Enable Asset Monitoring**; click **Save Asset Identity**. **Expected:** Network status changes to Enabled/Connected or an explicit error; secrets are never returned, only configured booleans. Native disables Contract/Mint; token without contract is rejected. Test without endpoint returns provider endpoint-not-configured. EVM + **Finalized** is rejected server-side with `UNSUPPORTED_FINALITY_POLICY` (`admin-blockchain-monitor-config.tsx:253-367`; `routes/blockchain-monitoring.ts:293-330`).

C17 - Blockchain readiness blocked-to-ready edge cases

Role/route: Monitoring operator, network drawer. **Preconditions:** Existing monitor with incomplete or stale status. **Steps:** 1. Inspect `Readiness` and each reason. 2. Disable network or asset, remove endpoint, choose incompatible Adapter/Provider, or let health become stale. 3. Correct endpoint/compatibility, enable both, save/test, save identity again. 4. Reopen drawer. **Expected:** Badge is `BLOCKED` with reasons such as `Network monitor disabled`, `Endpoint secret not configured`, `Incompatible provider/adaptor combination`, `Provider disconnected`, `Health check stale`, `Asset identity incomplete`; only current connected health/proof and exact route readiness produce `READY` (`admin-blockchain-monitor-config.tsx:33-75,195-247`; `routes/blockchain-monitoring.ts:136-219`).

C18 - Blockchain setup ready-route enablement and address/memo validation

Role/route: Monitoring operator, blockchain setup utilities/API. **Preconditions:** Manual deposit route with receiving address; monitor setup endpoint. **Steps:** 1. Review setup routes and missing configuration. 2. Correct receiving address and required memo/tag. 3. Use enable-ready/selected operation for one exact asset-network ID. 4. Include Bitcoin route

as a negative case. **Expected:** Only routes with valid address/memo, compatible provider, current network health proof and exact identity proof become enabled; skipped routes report reasons. Bitcoin is not auto-enable eligible; invalid token/contract or memo yields blocked/incomplete, not partial activation (`routes/blockchain-monitoring.ts:128-219,230-267`).

C19 - Blockchain match review approve/reject and registration gap

Role/route: Authorized monitoring/order operator, monitoring matches/gaps. **Preconditions:** Match is `needs_review`. **Steps:** 1. Open match evidence. 2. Attempt **Approve** with stale/noncanonical/nonfinal evidence. 3. Refresh evidence, then approve. 4. For an invalid payment choose **Reject**, provide reason. 5. For a registration gap choose activate and inspect resulting watch. **Expected:** Approval requires fresh canonical finalized evidence; otherwise `BLOCKCHAIN_EVIDENCE_NOT_FINAL`. Valid approval transitions order appropriately and competing candidates are rejected. Rejecting funded evidence returns order to awaiting funds and creates a fresh registration gap; activation requires an active watch from current chain head (`routes/blockchain-monitoring.ts:354-453`).

C20 - Workspace config sync preview/apply and stale-hash protection

Role/route: Owner only, Admin overview → **Workspace configuration sync**. **Preconditions:** JSON snapshot containing catalog/pricing/site data; current configuration unchanged. **Steps:** 1. Import JSON via file picker. 2. Click preview. 3. Inspect categories/counts for `add`, `update`, `softDisable`, `unchanged`, object availability/copy-on-apply and customer-deposit eligibility. 4. Apply reviewed changes and accept confirmation text. 5. Repeat after changing live config between preview/apply. **Expected:** Preview is dry-run and no writes occur. Apply requires exact `WORKSPACE_CONFIG_APPLY` and nonempty `expectedStateHash`; stale state returns `WORKSPACE_CONFIG_CHANGED` and demands fresh preview. Removed production-only rows are soft-disabled, not deleted. Invalid references, invalid pricing tiers, bad object hashes/content types return `INVALID_WORKSPACE_CONFIG`; eligibility determinant changes force Customer Deposits OFF (`workspace-config-sync.ts:66-85,127-190,262-318,321-341,345-387`).

C21 - Permission matrix and forged-owner-key negative case

Role/route: Staff vs owner, all routes. **Preconditions:** Staff authorization with selected effective permissions; owner authorization. **Steps:** 1. As staff with `catalog.view` only, open currencies and attempt create/edit/delete/bulk. 2. As staff with `pricing.view` only, open pricing and try save/bulk. 3. As staff with `integrations.view`, try credential create/update/rotate and WhiteBIT permission verification. 4. As non-owner with forged owner-only effective keys, repeat. 5. As owner, repeat config sync, receiving-wallet and credential actions. **Expected:** Read controls appear only with view permission; mutations are hidden/disabled or rejected server-side without manage permission. Owner-only keys include `crypto_networks.receiving_wallets`, `receiving_wallets.manage`, `integrations.credentials.create/update/delete/rotate` and team lifecycle keys; a forged key cannot grant staff access. Owners bypass permission checks (`admin-permissions.tsx:8-29,45-64`; route shells in `admin.tsx:1275-1278,9436`).

Admin content, editorial, branding and team

Inventory (implemented surfaces and evidence)

- **Site content:** `artifacts/crypto-exchange-widget/src/pages/site-content.tsx (/admin/site-content)`. Page tabs are derived from `SITE_PAGE_KEYS / PAGE_LABELS`; editor supports `Visual Editor` and `Advanced JSON`, per-page draft editing, JSON validation, SEO/navigation fields, header/footer visibility, hero/content sections, CTA labels/URLs, widget exchange information, navigation/partner-logo areas, social/trust editing and contact inbox/preview. `Publish` publishes a section/snapshot; `Save draft` controls are distinct. `Publish Snapshot` is owner-only and requires publication changes to be present.
- **Blog:** `admin-blog.tsx (/admin/blog)`, `admin-blog-editor.tsx (/admin/blog/new, /admin/blog/edit/:id)`, and `admin-blog-automation.tsx (/admin/blog/automation)`. List supports search, statuses `Published`, `Scheduled`, `Draft`, actions `Edit Article`, `View Public`, `Publish Now`, `Revert to Draft`, `Delete`; editor supports HTML/JSON Blocks, categories, statuses `Draft`, `Published`, `Scheduled`, `Unpublished`, tags, citations, images, SEO, author/meta, featured flag and schedule timestamps. Automation supports RSS/Atom/CoinMarketCap sources, reliability, enabled state, settings, `Preview Run`, `Run Now`, and `Save Settings`.
- **Newsletter:** `admin-newsletter.tsx (/admin/newsletter)`. Send an important update composer has `Update title`, `Short description`, `Read More link` (e.g. `/blog/your-article`), `Publish update`; subscriber list has `Refresh`, statuses `active/disabled`, `Disable/Enable`, `Remove` with confirmation.
- **Appearance:** `admin-appearance.tsx (/admin/appearance)`. Brand assets `Light Mode Logo (Required)`, `Dark Mode Logo (Required)`, `Mobile Icon (Optional)`, `Favicon (Optional)`; `Choose File`, `Clear`, drag/drop; live Desktop/Tablet/Mobile previews; responsive size sliders/number inputs, `Reset Logo Sizes`, `Alignment Left / Center / Right`, source dimensions and `Save Changes`, reset-to-default control when custom branding exists.
- **Background:** `admin-landing-background.tsx (/admin/landing-background)`. Premium Presets / Custom Upload, ten actual preset IDs, desktop/mobile placement, horizontal/vertical, zoom, opacity, blur and `Reset Blur`, focal map drag, preview, publish. Custom upload accepts JPEG/PNG/WEBP, non-empty, max 10 MB. A non-manager sees `You have view-only access to landing settings`.
- **Team:** `admin-team.tsx (/admin/team; /admin/staff is legacy/redirect in current App routing)`. Tabs `Members`, `Roles`, `Activity`, individually permission-gated. `Members` supports `Invite Member`, paging/per-page, bulk `Enable`, `Disable`, `Edit`, `Delete`, roles/overrides, suspend/reactivate/remove. `Roles` supports `Create Role`, permission catalog, edit/delete with usage protection. `Activity` has operator/action/target/date and `Previous / Next` cursor paging.
- **Authorization/MFA/shell:** `src/lib/admin-permissions.tsx` explicitly prevents forged owner-only permissions for non-owners; `App.tsx` defines authorization gates, MFA required/enrollment-required states, routes and `AdminShell`. Admin nav labels include Overview, Orders, Revenue, Customers, Affiliates, Payouts, Program Settings, Appearance, Providers, Background Studio, API Integrations, Currencies & Methods, Manual Pricing, Staff / Team, Site content, Blog, Newsletter Subscribers. Relevant e2e: `e2e/admin-staff.spec.ts`, `e2e/admin-authorization.spec.ts`, `e2e/landing-background.spec.ts`, and shared shell/responsive assertions in `e2e/responsive-layout.spec.ts`.

SC-01 - Admin shell menu, route titles, and responsive navigation

Role/route: approved owner/operator; `/admin` and each listed Admin route. **Preconditions:** authorization API returns an active approved operator; desktop and phone viewports available. **Steps:**

1. Open `/admin` and inspect the left navigation and shared header.

2. Select Site content, Blog, Newsletter Subscribers, Appearance, Background Studio, and Staff / Team one at a time.
3. Confirm each URL and page title/eyebrow; open the profile/notifications controls.
4. At a phone width, open the Admin mobile menu and select the same destinations.
5. Visit legacy /admin/staff if exposed by a bookmark.

Expected (observable): shared .admin-shell, sidebar/header, route-specific title and section label render; menu links navigate to /admin/site-content, /admin/blog, /admin/newsletter, /admin/appearance, /admin/landing-background, /admin/team; phone menu is usable without document overflow; legacy staff entry resolves to Team (current App includes redirect). No duplicate title/header or clipped controls. responsive-layout.spec.ts asserts shared headers, mobile menu, touch targets and no overflow.

SC-02 - Site content page selection and visual draft editing

Role/route: operator with site_settings.view and site_settings.manage; /admin/site-content. **Preconditions:** draft site content loads successfully. **Steps:**

1. Open the page and select each visible tab in Editable public pages.
2. Confirm the heading changes to <page> draft.
3. Choose Visual Editor.
4. For a page with a hero, change Headline / Title, Subtitle / Intro, Main page text, hero image path/alt text, primary/secondary button labels and URLs.
5. Toggle Show in Header and Show in Footer and change SEO Title, SEO Description, and Navigation label.
6. Select Save draft and reload the same page.

Expected: only the selected page draft is edited; actual controls retain values after save/reload; the draft is not assumed public merely because it was saved; success notice is observable and loading disables the save control. Other page tabs remain unchanged.

SC-03 - Site content Advanced JSON validation and publish boundary

Role/route: content manager/owner; /admin/site-content. **Preconditions:** a loaded page draft. **Steps:**

1. Choose Advanced JSON and verify the page-specific JSON textarea label (<Page> draft JSON).
2. Replace the value with malformed JSON and attempt the relevant save action.
3. Replace it with valid JSON containing a changed title/body and save the draft.
4. Select the section Publish control (button-publish-site-section) and observe the notice.
5. Change another draft field and inspect the Publish Snapshot action in the navigation/partner-logo publication area.
6. As a non-owner with management permission, attempt to use snapshot publication.

Expected: malformed JSON produces a visible validation/error notice and no save request; valid JSON saves only a draft; Publish reports a successful published revision when the API succeeds; a changed draft does not become public until publish; owner-only snapshot publication is hidden/disabled for non-owner. API failure leaves the draft visible and reports an error rather than silently claiming publication.

SC-04 - Blog list search, status actions, and destructive safety

Role/route: blog manager with blog.view/blog.manage; /admin/blog. **Preconditions:** list contains published, scheduled, draft and/or unpublished records. **Steps:**

1. Verify columns Article, Status, Published/Scheduled, Actions.

2. Type a title fragment and then a slug fragment into `Search articles...`; clear it and enter a nonmatching value.
3. Open `Article actions` for a published article and choose `View Public`; separately choose `Revert to Draft`.
4. Open actions for a draft/scheduled article and choose `Publish Now`.
5. Choose `Delete`, cancel the browser confirmation, then repeat and confirm.

Expected: filtering is case-insensitive over title and slug; no match displays `No articles found.`; published records show `View Public` and `Revert to Draft`, nonpublished records show `Publish Now`; delete is protected by the exact confirmation text and cancellation makes no request; successful mutations refresh the list. Failed mutation does not fabricate a changed status.

SC-05 - Create blog article draft with required fields and body formats

Role/route: blog manager; `/admin/blog/new`. **Preconditions:** at least one category is available. **Steps:**

1. Open `New Article`; verify `Title`, `Slug`, `Excerpt`, `Content Body`, `Category`, `Status`, `Tags` (Comma-separated), `Reading time` (minutes), and `Featured article`.
2. Click `Save Article` with title, slug, or category missing.
3. Enter a title and verify slug auto-generation while it still matches the prior generated slug; manually customize slug and change title again.
4. Enter HTML in the body with `HTML` selected and save a `Draft`.
5. Edit the article, select `JSON Blocks`, enter malformed JSON, then save; correct it to valid JSON and save.

Expected: missing required fields shows `Title, slug, and category are required`; generated slug follows the title until manually changed; custom slug is preserved; HTML is saved as text; malformed blocks show `Invalid JSON for blocks format` and do not submit; valid blocks save and route changes to `/admin/blog/edit/:id` with `Article created successfully`.

SC-06 - Blog publishing metadata, citations, media, and SEO

Role/route: blog manager; `/admin/blog/edit/:id`. **Preconditions:** existing article loads; upload intent and object-storage PUT can be stubbed. **Steps:**

1. Set `Status` to `Published`, `Scheduled`, `Unpublished`, and `Draft` as applicable; set `Published At / Scheduled At` where appropriate.
2. Add a citation with `+ Add Citation`; fill `Source URL`, `Source Title`, `Publisher`, date and `Claim/Quote`; remove it with the citation `×` control.
3. Use `Click to upload featured image` and `Click to upload social image`; verify previews and `Remove` controls.
4. Fill `Image Alt Text`, `SEO Title`, `SEO Description`, `Canonical URL`, `Author Name`; toggle `Index Page` and `Follow Links`; toggle `Featured article`.
5. Save and reload.

Expected: all fields round-trip in the update payload; uploaded images show previews and object paths; an upload failure shows `Failed to upload image` and does not set a broken path; `Author ID` remains disabled/Managed by server; removed citations/images are absent after save; status and schedule values are observable after reload.

SC-07 - Blog optimistic-concurrency and load/API errors

Role/route: blog manager; `/admin/blog/edit/:id`. **Preconditions:** article has an `updatedAt` value. **Steps:**

1. Change a field and intercept update to return HTTP 409.
2. Save the article.

3. Reload an unknown ID; separately make the article GET fail.
4. Make article list GET fail and open `/admin/blog`.

Expected: update includes `expectedUpdatedAt`; 409 shows `Conflict: This article was modified by another user or process. Please refresh and review changes before saving again.`; unknown article shows `Article not found`; list failure shows `Failed to load articles`; loading states are shown while requests are pending and no false success is shown.

SC-08 - Blog automation source lifecycle and validation

Role/route: blog manager; `/admin/blog/automation`. **Preconditions:** automation page loads; source list may be empty. **Steps:**

1. In `Add New Source`, submit with missing `Name` or `URL`.
2. Fill both, choose each actual `Source Type` (`RSS`, `Atom`, `CoinMarketCap`), choose `Reliability` (`Standard`, `Reliable`, `Official`), toggle `Enabled`, and click `Add Source`.
3. Choose source `Edit`, change values, click `Update Source`, then click `Cancel` while editing.
4. Choose delete, cancel `Remove this source?`, then confirm.

Expected: missing fields show `Name` and `URL` are required; successful create/update notices are `Source added`/`Source updated`, form resets, and list refreshes; cancel preserves server data; delete cancellation is safe, confirmation removes the source, and API failure shows the exact failure notice.

SC-09 - Blog automation settings save, preview, and run safety

Role/route: blog manager; `/admin/blog/automation`. **Preconditions:** settings load. **Steps:**

1. Toggle `Enable Automation`; set `Cadence Unit` to `Daily/Weekly`, `Articles Per Period`, `Timezone`, `Schedule Times`, `Publication Mode`, `Topics`, `Categories`, `Keywords`, `Language`, and `Min Length`.
2. Toggle `Require Multiple Sources`, `Generate Featured Image`, and `Generate SEO Meta`.
3. Click `Save Settings`, reload and verify comma-separated values persist as arrays.
4. Click `Preview Run`; inspect `Preview Result JSON`.
5. Click `Run Now`, cancel `Run the blog automation now?`, then repeat and confirm.

Expected: `Save Settings` sends trimmed comma-separated entries and shows `Settings updated successfully`; failures show `Failed to update settings`; preview does not trigger a run and renders the result; run confirmation prevents cancellation; confirmed run reports returned status, while failure alerts `Failed to run automation`.

SC-10 - Newsletter announcement validation and queueing

Role/route: site settings manager; `/admin/newsletter`. **Preconditions:** `site_settings.manage` and subscriber data available. **Steps:**

1. In `Send an important update`, submit empty fields.
2. Fill `Update title`, `Short description`, and `Read More link` (e.g. `/blog/your-article`) with a non-URL/non-slash value and submit.
3. Use an absolute `https://` link and then a site path such as `/blog/your-article`; click `Publish update`.
4. While publishing, click the button again.

Expected: empty submission shows `Enter a title, short description, and published page link.`; invalid link shows `The Read More link must be an absolute URL or a site path beginning with /.`; valid submission queues for active subscribers, clears fields, and shows `Announcement queued for active subscribers.`; pending state prevents duplicate submission; API failure shows `Unable to publish announcement.`

SC-11 - Newsletter subscriber enable/disable/remove and read-only controls

Role/route: settings manager or view-only operator; `/admin/newsletter`. **Preconditions:** active and disabled subscribers exist. **Steps:**

1. Confirm email and `Subscribed <date>` display plus `active/disabled` status.
2. For active subscriber click `Disable`; for disabled click `Enable`; refresh the list.
3. Click `Remove`, cancel `Remove this subscriber?`, then repeat and confirm.
4. Force list GET failure and use `Refresh`.
5. Open as a user without `site_settings.manage`.

Expected: active exposes `Disable`, disabled exposes `Enable`; successful mutation refreshes status; remove cancellation is safe and confirmation removes the subscriber; load failure says `Unable to load subscribers`. `Refresh` and try again.; `Refresh` is disabled while fetching; view-only users can see list but cannot see composer/mutation controls.

SC-12 - Appearance assets, file safety, and logo previews

Role/route: branding manager with site settings access; `/admin/appearance`. **Preconditions:** branding loads. **Steps:**

1. Verify required/optional labels and Desktop/Tablet/Mobile live previews.
2. Choose or drag/drop JPEG, PNG, WEBP and SVG files into light/dark/mobile/favicon uploaders.
3. Try a file over 5 MB and an unsupported MIME type.
4. Clear `Mobile Icon (Optional)` and `Favicon (Optional)` where present.
5. Confirm preview updates immediately, then click `Save Changes`; reload.

Expected: accepted file shows upload progress (`Requesting upload...`, `Uploading image...`, `Upload success`) and preview; oversized file reports `File too large (max 5 MB)`; unsupported file reports `Invalid file type`; optional clear sets null; save is disabled when unchanged, reports `Brand assets saved successfully`. on success, and persisted assets/sizes reload. Upload or save failure reports an error without replacing the prior saved asset.

SC-13 - Appearance responsive sizing, alignment, reset, and API load failure

Role/route: branding manager; `/admin/appearance`. **Preconditions:** branding data loaded. **Steps:**

1. Adjust Desktop/Laptop, Tablet/iPad and Mobile `Logo Width` and `Logo Max Height` sliders and number inputs.
2. Enter below-minimum and above-maximum values; select `Left`, `Center`, and `Right` Alignment.
3. Click `Reset Logo Sizes`, verify only responsive defaults change, then save.
4. Change `Source Width (px)`, `Source Height (px)`, and `Legacy Display Max Width (px)` and save.
5. Make branding GET fail and click the retry control.

Expected: number controls clamp responsive values to their documented ranges; previews reflect each device; alignment selection is visible; reset restores defaults without silently saving; successful save invalidates admin/public branding queries; load failure shows `Failed to load branding`. and exposes retry; server error uses `Failed to save brand assets`. unless a more specific API message is supplied.

SC-14 - Background preset selection, placement editing, and device separation

Role/route: site settings manager; `/admin/landing-background`. **Preconditions:** current background loads; manager has `site_settings.manage`. **Steps:**

1. Confirm `Premium Presets` and `Custom Upload` tabs and select each of the ten visible presets, ending on a different preset.
2. In placement controls choose `Desktop`, set Horizontal to `Left / Center / Right`, Vertical to `Top / Center / Bottom`, and adjust `Zoom`, `Opacity`, and `Blur`.

3. Drag the focal point on `focal-map`; click `Reset Blur`.
4. Switch to `Mobile`, make different placement changes, switch back to `Desktop`.
5. Inspect desktop/mobile previews and click the publish action.

Expected: selected preset is visibly marked; values remain bounded (x/y 0-100, zoom 100-150%, opacity 0-100%, blur 0-30px); desktop and mobile placement values remain independent; focal drag clamps within map; reset sets blur to 0px; publish sends normalized placements and reports the localized publish success notice; unchanged draft does not needlessly publish.

SC-15 - Background custom upload validation, publish failure, and view-only safety

Role/route: manager and view-only operator; `/admin/landing-background`. **Preconditions:** background loads.

Steps:

1. Select `Custom Upload` and try an empty file, >10 MB file, SVG/GIF, and valid JPEG/PNG/WEBP.
2. During valid upload inspect immediate preview, `Requesting upload...`, `Uploading image...`, and final success state.
3. Make the storage PUT fail and inspect recovery.
4. Make publish return 403, then a generic API error.
5. Open as a user lacking `site_settings.manage`.

Expected: invalid files show localized empty/too-large/invalid-type errors and do not create a custom source; valid upload sets mode `custom` and stores object path; PUT failure reverts the unpublishable local preview and reports upload error; 403 reports the owners-only notice, other errors report publish failure; view-only sees exactly `You have view-only access to landing settings.` and cannot edit/publish.

SC-16 - Team members paging, invitation, and owner protection

Role/route: owner; `/admin/team` → `Members`. **Preconditions:** authorization returns owner; members/roles APIs return data. **Steps:**

1. Verify `Team Members`, columns `Operator/Status/Role/Actions`, `Invite Member`, `Per page`, and pagination.
2. Change per-page to 25/50/100 and navigate `Previous/page/Next`.
3. Click `Invite Member`; submit empty name/email, then valid `Name`, `Email`, and `Assigned Role (No specific role (No access) or a role)`; click `Send Invite`.
4. Select ordinary members and use bulk `Enable`, `Disable`, `Edit`, and `Delete`; cancel the delete confirmation and then confirm.
5. Inspect owner/current-member checkboxes and actions.

Expected: page range and rows update; required invite fields block invalid submission; successful invite closes dialog and refreshes; bulk toolbar shows `<n> selected`, uses correct status-specific operations, and reports partial failures; owner/current member/removed members cannot be selected; owner row is protected and cannot be suspended/removed.

SC-17 - Team member role/permission overrides and stale-version safety

Role/route: owner; `/admin/team` → `Members`. **Preconditions:** member has a role and authorization catalog. **Steps:**

1. Open row `...` → `Edit role & permissions`.
2. Select a different `Assigned Role`; observe overrides clear.
3. In `Permission Overrides`, inspect `Inherited`, `Allowed (Override)`, `Denied (Override)`, and `Owner Only`; toggle one allow and one deny.
4. Submit and reload the list.
5. Return a conflict/error from update and retry with stale `authVersion`.

Expected: dialog is scrollable and identifies the member; role change clears old overrides; owner-only catalog entries are not editable; selected overrides are mutually exclusive; update includes `expectedAuthVersion`; success closes and refreshes; conflict/error leaves the dialog open and does not claim saved permissions.

SC-18 - Team roles create/edit/delete and usage guard

Role/route: owner; `/admin/team` → Roles. **Preconditions:** roles include one used role and one unused role. **Steps:**

1. Click `Create Role`; enter `Role Name`, `Description`; inspect grouped `Permissions`, including `Owner Only`; submit `Create Role` or cancel.
2. Select a role and use bulk `Edit`; change permissions and submit `Save Role`.
3. Attempt to delete a role with `Usage > 0`, then delete an unused role; cancel and confirm the browser confirmation separately.
4. Change `Per page` and use pagination.

Expected: create/edit dialogs expose only catalog permissions; owner-only permissions are labels, not checkboxes; cancel causes no mutation; used-role delete is disabled/blocked with `Cannot delete <role> because it is assigned to <n> member(s).`; unused deletion requires confirmation and refreshes roles/members; table displays granted permission count and member usage.

SC-19 - Team activity log pagination and empty/error states

Role/route: operator with `team.activity.view`; `/admin/team` → Activity. **Preconditions:** activity API supports cursor pages. **Steps:**

1. Open `Activity` and inspect Operator, Action, Target, Date & Time.
2. Verify entity kind, shortened ID, safe label, and `System` for events without an entity.
3. Click `Next`, wait for the next cursor page, then click `Previous`; repeat at boundaries.
4. Return an empty page and then a load error.

Expected: 50-item cursor request is used; `Next` is disabled without `nextCursor`, `Previous` is disabled on first page, and fetching disables navigation; cursor history returns to the exact previous page; empty state says `No activity found.`; failure says `Could not load activity log`.

SC-20 - Permission-derived tabs/menu and forged owner-only key defense

Role/route: non-owner operators with narrowly scoped permissions; `/admin/team` and Admin routes. **Preconditions:** authorization response grants selected non-owner `effectivePermissions`, optionally including owner-only keys maliciously. **Steps:**

1. Grant only `team.members.view`; open `/admin/team`.
2. Grant only `team.roles.view` or only `team.activity.view` in separate sessions.
3. Attempt direct navigation to `/admin/blog/automation`, `/admin/blog/new`, `/admin/newsletter`, `/admin/landing-background`, and team mutation controls without corresponding manage/owner permission.
4. Include forged owner-only keys such as `team.members.invite`, `team.roles.delete`, or `integrations.credentials.create` in a non-owner authorization payload.

Expected: Team shows only permitted tabs and selects the first available tab; unauthorized controls/routes are hidden or render the shared authorization boundary; a non-owner cannot gain owner-only controls from forged `effectivePermissions`; direct API denial is presented as denied rather than bypassed client-side. `admin-permissions.tsx` explicitly hard-blocks the owner-only key set.

SC-21 - Admin authorization denial, backend outage, MFA challenge, and enrollment

Role/route: unauthenticated/denied operator; `/admin`. **Preconditions:** intercept authorization/summary APIs with each documented response. **Steps:**

1. Return genuine 403 `OPERATOR_ACCESS_DENIED` and navigate `/admin`.
2. Return 503 `OPERATOR_AUTH_UNAVAILABLE`, inspect the unavailable boundary, then restore 200 and click `Try again`.
3. Return authorization code `ADMIN_MFA_REQUIRED`; inspect `operator-mfa-challenge` and complete/submit the presented challenge or cancel.
4. Return `ADMIN_MFA_ENROLLMENT_REQUIRED`; inspect `operator-mfa-enrollment-required` and follow the presented enrollment action.
5. Repeat a protected route after MFA success/failure.

Expected: genuine denial does not expose Admin content and is not misreported as backend outage; outage says `Your account has not been denied.` and exposes `Try again`; retry reaches `Overview` after recovery; MFA-required and enrollment-required states render their dedicated test IDs/challenge or enrollment controls, keep protected content unavailable until satisfied, and show an error on failed challenge rather than granting access. `e2e/admin-authorization.spec.ts` covers approved access, genuine denial, outage distinction and retry; App contains both MFA branches.

SC-22 - Responsive content tables, editors, dialogs, and touch safety

Role/route: approved manager/operator; all listed routes. **Preconditions:** representative populated data; phone (375px), tablet and desktop viewports. **Steps:**

1. Open Blog list, Newsletter, Team Members/Roles/Activity, Site content and Background Studio at 375px.
2. Exercise horizontal table scrolling, search, tabs, editor controls, upload dropzones, focal map, and dialogs.
3. Open the Team `Edit Member Permissions`, role form, and Blog editor on a phone; scroll each dialog and submit/cancel.
4. Inspect document width and primary controls at phone/tablet/desktop.

Expected: tables remain contained in their scroll wrappers rather than causing document overflow; forms stack responsively; dialogs remain within viewport with internal scrolling; focal controls and previews remain usable; touch controls meet the shared 44px contract where asserted; no clipped `Save/Publish/Cancel` action or duplicate Admin shell. Shared responsive assertions are in `e2e/responsive-layout.spec.ts`, while Team phone table containment is explicitly exercised in `e2e/admin-staff.spec.ts`.

Telegram bot and Telegram Mini App

Surface inventory

Live Telegram bot (not the Mini App)

- Webhook/update handling is private-chat only (`artifacts/api-server/src/routes/telegram.ts`); chat identity is upserted and locale is derived from Telegram language (:185-190). Wizard sessions are persisted for 30 minutes and update-id aware (:192-200), with per-chat advisory locking (:161-166).
- Main menu buttons are: Exchange, Track, Orders, Sign In or My Account, Sign Up or Sign Out, Language, Support, Website (or Website unavailable), and optional Open App (:174-184). Sign-in/sign-up opens a website `/telegram/connect?token=...&intent=...` challenge (:206-217).
- Exchange wizard: source/target prompts, option labels `ASSET · NETWORK/TITLE`, search by typing, eight options per page, Back, Next , and Cancel Exchange (:237-315). Then amount side buttons are the source and target asset codes plus Cancel Exchange (:423-467).
- Optional field buttons include <option>, field pagination Back / Next, and Skip when optional (:393-403). Amount/quote logic supports entering send or receive amount, live limits/rates, retry/matching behavior (:471-525).
- Orders displays up to 10 records; linked accounts use customer history/website order URLs, anonymous users use signed-token status checks and Track <orderId> callback buttons (:528-560). Deposit instructions include address/photo and optional memo (:562-572). Status/creation notifications are outboxed and retried, with recovery after ambiguous create responses (:633-655, 678-700, 1220-1281).

QuickXchange Telegram Mini App (all routes)

- Routes: `/ Home`, `/exchange`, `/orders`, `/track`, `/orders/:id`, `/support`, `/account`; unknown route is Not Found (`artifacts/quickxchange-telegram-mini-app/src/App.tsx`). Every route is protected; outside Telegram it shows Open in Telegram and Please open the bot to access your account (`App.tsx:31-61`).
- Auth waits up to four seconds for `Telegram.WebApp.initData`, calls `ready()/expand()`, validates server-side, stores a 15-minute signed bearer token in `sessionStorage`, and fails closed on absent/invalid data (`src/lib/auth.tsx`; API `artifacts/api-server/src/routes/telegram-mini-app.ts`). Session requests require valid HMAC, nonfuture/nonexpired `auth_date`, positive user id, and existing matching Telegram chat (`telegram-mini-app.ts:93-100`).
- Bottom buttons are Home, Exchange, Orders, Support, Account; hidden on order detail and while unauthenticated (`src/components/layout.tsx:12-23`).
- Home offers Swap, Convert, View All, order cards, and empty-state Start an Exchange (`src/pages/home.tsx:58-76, 79-149`). Orders offers Track ID, Search orders..., order cards, and empty-state Start an Exchange (`src/pages/orders.tsx:54-85, 101-167`).
- Exchange is a three-step wizard: Swap/Convert, source/target selectors, amount, swap-direction icon button, step Continue, Back, details, review, and Place Order (the latter controls are in `exchange.tsx:571-607` and later render). It uses manual exchange quote/order APIs for Swap and Quickex quote/order APIs for Convert (`exchange.tsx:250-267, 340-543`).
- Order detail has Order Details, back arrow, conditional refresh icon, status timeline, Exchange Summary, copy controls, deposit/payment details, I Have Sent Crypto or Mark as Paid, Contact Support, Completed/Cancelled, and lower Refresh Status/order navigation controls (`src/pages/order-detail.tsx:321-340, 345-405, 448-517, 520-559`). Status polls every 3 seconds until terminal (:64-81).

- Track form labels are Order ID, Signed Tracking Token, button Track Order/Searching...; errors intentionally say Order not found or you do not have permission to view it. (src/pages/track.tsx:45-94). Account shows Connected ✓ or Link QuickXchange Account, Log In, Create Account, My Orders, Support (src/pages/account.tsx:60-141). Support button is Chat with Support; missing configuration says Support URL not configured. (src/pages/support.tsx:23-50).
- Mini-App API endpoints: session, list/detail orders, account-link challenge, and signed-token order link (telegram-mini-app.ts:219-328). Orders are scoped to the Telegram chat; linking validates order id and signed tracking token and claims unowned orders for a linked customer, rejecting ownership conflicts (:306-346). Projections expose safe deposit/payment/settlement fields and verified funding transaction (:102-176,124-155).

Live Telegram bot scenarios

BOT-01 - Open bot menu as a new private-chat user

Role/route: Anonymous Telegram user; live bot /start/main menu. **Preconditions:** Bot configured; private Telegram chat; no linked customer account. **Steps:** 1. Send /start (or invoke the bot entry point). 2. Inspect the inline keyboard. 3. Tap each menu button without starting an exchange. **Expected (observable):** Welcome/choose text appears; buttons are exactly < Exchange, 📦 Track, 📋 Orders, 👤 Sign In, 📝 Sign Up, 🌐 Language, 💬 Support, 🌍 Website, and, when configured, 📱 Open App. Sign In/Sign Up produce a website button, not an unauthenticated account disclosure. References: telegram.ts:174-217.

BOT-02 - Reject non-private or malformed update

Role/route: Security/permissions; webhook update handling. **Preconditions:** Bot webhook reachable. **Steps:** 1. Deliver a group/supergroup update. 2. Deliver a callback update with no numeric Telegram sender. 3. Deliver a private update with missing chat id. **Expected:** Updates are not treated as a customer session or wizard action; no order/account data is exposed. Reference: telegram.ts:67-68,168-172.

BOT-03 - Start Swap wizard and select source/target

Role/route: Anonymous or linked user; Exchange -> Swap. **Preconditions:** Exchange config contains at least two valid manual routes. **Steps:** 1. Tap < Exchange, choose Swap if prompted. 2. Confirm source prompt says Which currency or payment method do you send?. 3. Tap an option. 4. Confirm target prompt says Which currency or payment method do you receive?. 5. Tap a valid target. **Expected:** Options display as ASSET · NETWORK/TITLE; selected source/target confirmations show ✓ You Send and ✓ You Receive; only configured route-compatible targets are offered; amount-side choice follows. References: telegram.ts:245-253,318-391,423-469.

BOT-04 - Search, paginate, cancel, and expire wizard

Role/route: Wizard negative/edge; source/target selection. **Preconditions:** More than eight options; active wizard. **Steps:** 1. Type a distinctive asset/payment name. 2. Verify Results for: and matching options. 3. Tap Next ➡, then ⬅ Back. 4. Tap ✗ Cancel Exchange. 5. Leave another wizard idle for over 30 minutes, then tap an old callback. **Expected:** Search filters options; max eight option rows are shown; page controls work; cancel abandons the flow; expired/no-session callbacks do not continue or create an order. No-match text says No available options match your search. References: telegram.ts:255-315,198-200.

BOT-05 - Enter amount on send side and validate limits

Role/route: Swap/Convert amount stage. **Preconditions:** Valid source/target selected; configured min/max. **Steps:** 1. Tap the source asset-code amount-side button. 2. Enter below minimum, exactly minimum, above maximum, and a valid amount. 3. Repeat using the target asset-code button. **Expected:** Rate and Min / Max are shown; invalid values are rejected with an explanatory message; valid send value yields quote; target-side entry attempts to solve for a source amount and either matches tolerance or reports The live quote could not match the requested receive amount. Please try again. References: telegram.ts:442-467,489-525; amount validation helpers :405-415.

BOT-06 - Complete optional/conditional fields and required-field failures

Role/route: Wizard details stage; permissions/data validation. **Preconditions:** Chosen route has required and conditional settlement fields. **Steps:** 1. Provide an optional field via ⏭ Skip. 2. Select a field option (♦ ...). 3. Leave a visible required field empty or enter invalid numeric/pattern data. 4. For a conditional field, choose both matching and

nonmatching controlling values. **Expected:** Optional field can be skipped; field options and pagination work; invalid/empty visible required values prevent quote/order creation; conditional fields are requested only when their condition matches. References: `telegram.ts:393-415`; session sanitization `:192-196`.

BOT-07 - Create order and verify idempotent/recoverable behavior

Role/route: Swap or Convert order creation. **Preconditions:** Valid quote, details, email/address where required. **Steps:** 1. Submit a valid order. 2. Simulate timeout/5xx after provider may have created the order. 3. Retry the same update/request. 4. Inspect bot confirmation and order list. **Expected:** Exactly one order link is persisted; creation/status outbox records exist; bot eventually shows order id/status and deposit instructions where applicable. A 4xx is review/non-retry; transient/429/408 paths remain recoverable; ambiguous create is searched by client request id rather than duplicated. References: `telegram.ts:633-655,1218-1281` and retry decision search locations around `:91-101`.

BOT-08 - Orders, tracking callback, deposit, and status notifications

Role/route: Anonymous and linked users;  Orders,  Track `<orderId>`. **Preconditions:** At least one linked order in pending/processing/completed states. **Steps:** 1. Tap  Orders. 2. Verify displayed status. 3. Tap  Track `<orderId>`. 4. For a crypto/convert order, wait for deposit details. 5. Advance provider status through confirming, processing, completed, failed/cancelled. **Expected:** Anonymous history uses signed tracking token; linked history uses customer-owned history and website order links. Status labels update without exposing another user's order; deposit address and optional memo are delivered; lifecycle notifications are deduplicated by status version and terminal completion is acknowledged. References: `telegram.ts:528-572,1291-1353`.

BOT-09 - Account linking, sign-out, language, support, and unavailable website

Role/route: Account/auth edge. **Preconditions:** Both linked and unlinked Telegram chats; test once with website/support env configured and once without. **Steps:** 1. Tap  Sign In; follow  Sign In on website. 2. Repeat with  Sign Up and  Sign Up on website. 3. Return to bot and inspect linked menu. 4. Tap  Sign Out,  Language,  Support. 5. Unset website URL and repeat. **Expected:** Challenge URL contains a one-time token and intent; linked menu changes to My Account and Sign Out; sign-out removes linkage/session as designed; language selector changes locale; missing website yields Website unavailable and linking says Website account linking is temporarily unavailable. References: `telegram.ts:174-217`.

Mini App scenarios

APP-01 - Authenticate in Telegram and fail closed outside Telegram

Role/route: Mini App auth; `/` or any protected route. **Preconditions:** Open app via  Open App in Telegram; also test ordinary browser. **Steps:** 1. Open in Telegram and wait for initialization. 2. Confirm loading text. 3. Open the same URL outside Telegram, with no `initData`. 4. Replay an expired, future-dated, tampered-hash, malformed-user, or mismatched chat init payload. **Expected:** Telegram view calls `ready/expand`, applies theme, creates a 15-minute bearer session, and renders Home. Non-Telegram/invalid cases show Open in Telegram and Please open the bot to access your account; server returns 401-style auth error and does not create session. References: `auth.tsx:25-43,96-147`; `telegram-mini-app.ts:38-66,219-271`; `App.tsx:31-61`.

APP-02 - Verify Home, navigation buttons, empty state, and polling

Role/route: Authenticated Telegram user; `/`. **Preconditions:** Valid session with zero orders, then active and terminal orders. **Steps:** 1. Confirm greeting Hi, `<name>` and Welcome to QuickXchange. 2. Tap Swap, Convert, View All, and order cards. 3. With no orders, tap Start an Exchange. 4. With an active order, leave Home open for 3+ seconds; repeat after terminal status. **Expected:** Correct route opens; bottom buttons are exactly Home, Exchange, Orders, Support, Account; empty state says No recent orders yet; active orders refresh every 3 seconds and terminal lists stop polling. References: `home.tsx:42-149`; `layout.tsx:12-23`; `home.tsx:25-40`.

APP-03 - Swap three-step wizard with route search and reversal

Role/route: Authenticated user; `/exchange?mode=swap`. **Preconditions:** Exchange config includes manual routes and at least one crypto target/payment method. **Steps:** 1. Tap Swap. 2. Enter amount (default is 100). 3. Open source selector and target selector; search names and use All, Crypto, and Fiat filters if displayed. 4. Tap swap-direction icon. 5. Confirm live rate and click Continue. 6. On Details, click Back, alter route/amount, and continue again. **Expected:** Only route-compatible targets appear; source/target selectors, You Send, You Receive, live Fetching

rate... , and Back are visible; reversing swaps selections and clears quote/detail state; amount changes invalidate old quote. References: exchange.tsx:43-64,80-217,219-245,251-267,571-607,642-689 .

APP-04 - Swap validation, details, refund, and review

Role/route: Swap wizard edge/negative; /exchange?mode=swap . **Preconditions:** Quote with destination/settlement requirements. **Steps:** 1. Enter zero, below-min, and above-max amounts. 2. Continue with crypto target and blank destination address. 3. Leave conditional/required settlement field blank. 4. Enter refund address with a memo-required source but no refund memo. 5. Enter malformed email. 6. Supply all valid values and continue to Review. **Expected:** Errors are observable as Minimum amount is ... , Maximum amount is ... , Destination address is required , <field label> is required , Refund memo is required if refund address is provided , and Enter a valid customer email ; no step advance/order occurs until fixed. Details then Review show submitted values and Place Order . References: exchange.tsx:340-465 .

APP-05 - Convert live quote, expiry, refresh, and order creation

Role/route: Convert wizard; /exchange?mode=convert . **Preconditions:** Quickex config has a supported pair; provider can return quote. **Steps:** 1. Tap Convert . 2. Choose supported crypto source/target and amount. 3. Wait for debounced quote. 4. Verify receive amount and expiry countdown. 5. Let it expire; click Refresh quote . 6. Continue through Details/Review and click Place Order . 7. Refresh/double-submit during creation. **Expected:** Quote uses Quickex , waits/debounces , and shows Quote expired plus Refresh quote ; expired quote cannot continue or submit; refresh preserves route/amount; successful order links to Telegram chat and routes to /orders/<id> ; duplicate client request does not create duplicates. References: exchange.tsx:296-338,362-371,471-536 ; link API telegram-mini-app.ts:306-323 .

APP-06 - Conditional Convert fields and required destination inputs

Role/route: Convert Details; negative/edge. **Preconditions:** Quote returns required destination memo and conditional settlement fields. **Steps:** 1. Leave destination address blank and click Continue . 2. Add address but omit required memo. 3. Select a controlling field value that activates a conditional field and leave that field blank. 4. Select a value that disables it. 5. Enter valid email and all currently visible fields. **Expected:** Errors say Destination address is required , Destination memo is required , or the exact field label plus is required ; disabled/nonmatching conditional fields are not required; valid data advances to Review. References: exchange.tsx:399-465 and :274-292 .

APP-07 - Orders list search and order scoping

Role/route: Authenticated user; /orders . **Preconditions:** Multiple own orders across statuses plus another Telegram user's order. **Steps:** 1. Tap Orders then Track ID . 2. Search by partial id, source/target asset, and raw status. 3. Search a nonexistent term. 4. Attempt direct /orders/<other-user-id> . 5. Observe an active order for 3+ seconds then terminalize it. **Expected:** My Orders , Track ID , and Search orders... are visible; matching count appears; empty result says No matching orders ; foreign direct detail shows Order Not Found/no access; active polling stops at terminal. References: orders.tsx:54-85,101-168 ; API scoping telegram-mini-app.ts:198-217,274-292 .

APP-08 - Track an existing order with signed token

Role/route: Authenticated user; /track . **Preconditions:** Order ID and its signed tracking token from confirmation. **Steps:** 1. Open Track ID . 2. Verify labels Order ID , Signed Tracking Token , and placeholder Paste the token from your order confirmation . 3. Submit blank one/both fields. 4. Submit valid id + valid token. 5. Submit wrong token, altered token, nonexistent id, and another user's valid token. **Expected:** Track Order stays disabled until both fields are nonblank; valid pair links and navigates to detail; all invalid/unauthorized cases show exactly Order not found or you do not have permission to view it . and do not reveal which check failed. References: track.tsx:20-43,57-94 ; server validation telegram-mini-app.ts:306-326 .

APP-09 - Detail status timeline, refresh, polling, and terminal states

Role/route: Authenticated user; /orders/:id . **Preconditions:** Own pending, confirming/detected, processing, completed, failed/cancelled/expired orders. **Steps:** 1. Open an order card. 2. Observe Current Status , Order ID , timeline, Exchange Summary , You Send , You Receive . 3. Tap copy order ID and any address. 4. Tap refresh icon / lower refresh control. 5. Advance status through each lifecycle stage and reload. 6. Use back arrow and Telegram back button. **Expected:** Manual timeline labels are Created , Detected , Processing , Done ; convert labels use Created , Confirming , Processing , Done ; status text and color change; polling is 3 seconds only while nonterminal; completed shows Completed , failed/cancelled shows terminal panel and no refresh action; copied values visibly switch to check/Copied ; back returns /orders . References: order-detail.tsx:47-81,121-165,184-203,275-339,531-559 .

APP-10 - Crypto deposit/payment details and mark-paid permissions

Role/route: Pending manual/convert order detail. **Preconditions:** Pending crypto deposit with address/memo; pending fiat-payment order; order without payment details; already marked-paid or terminal order. **Steps:** 1. Verify `Crypto Deposit Details`, `Send Exactly`, network badge, address, optional `Memo / Tag (Required)`, QR code, and network-loss warning. 2. Tap address/memo copy buttons. 3. Tap `I Have Sent Crypto`. 4. For fiat, verify `Payment Details` and tap `Mark as Paid`. 5. Repeat after marked-paid and on completed/failed order. 6. Use `View on Explorer` when verified funding transaction exists. **Expected:** Exact amount/network warning is visible; copy feedback appears; mark-paid is available only while pending, payment details apply, and details exist; success updates state and disables duplicate action; terminal orders expose no payment action; transaction shows `Transaction ID`, `Copy`, `View on Explorer`, network and confirmations. References: `order-detail.tsx:167-177,216-257,448-517`; safe API projection `telegram-mini-app.ts:102-155`.

APP-11 - Missing payment details and support fallback

Role/route: Order detail/support edge. **Preconditions:** Nonterminal order with no currently available payment details; test configured and absent support URL. **Steps:** 1. Open detail. 2. Inspect payment panel. 3. Tap `Contact Support` when configured. 4. Open bottom `Support` and tap `Chat with Support`; repeat with support URL missing. **Expected:** Panel says `Payment details are not available yet.`; configured case has `Contact Support` and opens Telegram/external URL; Support page says `24/7 Support`; missing case disables button and displays `Support URL not configured`. References: `order-detail.tsx:475-487`; `support.tsx:23-50`.

APP-12 - Account linking and session-expiry recovery

Role/route: Authenticated unlinked/linked user; `/account`. **Preconditions:** Telegram session valid; test account link success, challenge failure, and session expiry. **Steps:** 1. Open `Account`; verify profile and `Link QuickXchange Account / Anonymous session`. 2. Tap `Log In`; repeat with `Create Account`. 3. Verify external `/telegram/connect` URL opens. 4. Complete link and reopen `Account`. 5. Expire/revoke bearer session, trigger an API call, then reload. **Expected:** Buttons are disabled while linking; challenge intent is `signin` or `signup`; success shows `Connected ✓` and `Verified account`; expired/revoked session clears token/user/link state and returns protected screens to `Open in Telegram` rather than showing stale orders. References: `account.tsx:9-37,60-117`; API `telegram-mini-app.ts:294-304`; auth expiry `auth.tsx:79-94,135-145`.

APP-13 - Unauthorized API and order-link ownership conflicts

Role/route: API/security test behind Mini App UI. **Preconditions:** Valid Mini-App token for user A; order owned by user B; missing/malformed bearer; valid tracking token for an order. **Steps:** 1. Call orders list/detail without `Authorization`. 2. Call with malformed/expired/tampered bearer. 3. Call account-link with intent other than `signin/signup`. 4. Call order-link with wrong tracking token, nonexistent id, and user-B-owned order. 5. Call valid link twice. **Expected:** Missing/invalid sessions return authentication failure; invalid intent returns `TELEGRAM_LINK_INVALID`; bad token returns `ORDER_TRACKING_INVALID`; nonexistent returns `ORDER_NOT_FOUND`; ownership conflict is rejected; repeat valid link is idempotent and does not duplicate relationship. References: `telegram-mini-app.ts:93-100,274-326`; `:294-304`; `:330-346`.

APP-14 - Direct routes, unsupported mode, and responsive controls

Role/route: Navigation/edge; all Mini App routes. **Preconditions:** Authenticated Mini App. **Steps:** 1. Navigate directly to `/exchange` without `mode`. 2. Use `?mode=gabrage`, then `?mode=swap` and `?mode=convert`. 3. Navigate to an unknown path. 4. Open `/orders/<id>` and inspect bottom navigation. 5. Test Telegram back button and narrow viewport. **Expected:** Missing/unknown mode defaults to `Convert` (`exchange.tsx:30-35`); explicit `Swap/Convert` selects correct wizard; unknown path renders `Not Found`; bottom navigation is hidden on order detail (`layout.tsx:20-23`); controls remain usable with safe-area padding and Telegram back returns to orders. References: `App.tsx:64-85`; `layout.tsx:60-71`; `order-detail.tsx:47-49`.

Operational APIs, monitoring and integrations

Inventory

- API route modules: `artifacts/api-server/src/routes/blockchain-monitoring.ts` (setup, registration gaps, watches/matches), `quickex.ts`, `whitebit.ts`, `notification-settings.ts`, `affiliate.ts`, `health.ts`, `workspace-config-sync.ts`, `customer-management.ts`, `exchange.ts`, `exchange-config.ts`.
- Contract surface: `lib/api-spec/openapi.yaml` documents `/healthz`, `/readyz`, `/quickex/*`, `/admin/quickex/*`, `/admin/whitebit/*`, `/webhooks/whitebit`, account order notifications, monitoring, deposit-provider reconciliation, referrals/affiliate, and config routes. An Admin-only `GET /admin/orders.xml` export is documented and implemented; no public RSS/Atom feed route was found. Static/public assets include widget `artifacts/crypto-exchange-widget/public/sitemap.xml` were found.
- Relevant tests: `manual-monitoring-readiness.test.ts`, `watch-registration-integration.test.ts`, `blockchain-monitoring-service.test.ts`, `quickex.test.ts`, `whitebit-integration.test.ts`, `whitebit-swap-integration.test.ts`, `whitebit-history-worker.test.ts`, `customer-status-notifications.test.ts`, `affiliate.test.ts`, `workspace-config-sync.test.ts`, `production-api-start.test.mjs`, `production-monitoring-migrations.test.mjs`, `customer-deposit-eligibility.test.ts`.
- Important gaps: no API XML feed endpoint; no explicit account-only Deposits decommission route (legacy order/deposit compatibility remains in `exchange.ts`, `order-history.ts`, and `schema/migrations`); no browser Admin screens are implemented in this API artifact, so scenarios below use exact API routes and identify expected UI labels only where routes map to known operator pages.

MON-01 - Readiness inventory before enabling customer deposits

Role/route: Owner/operator; `GET /admin/blockchain-monitoring/setup/routes`, `GET /admin/blockchain-monitoring/networks`, `GET /admin/blockchain-monitoring/assets/list`. Preconditions: authenticated operator; sandbox configuration containing BTC, Polygon token, TRON token, and BNB/BEP20 routes. Steps:

1. Call the three GET endpoints with operator auth.
2. Record each route's asset/network identity, adapter/provider, enabled state, health/proof timestamps, and customer-deposit state.
3. Compare a non-BSC route with a BEP20 route.

Expected (observable): response separates ready/unready routes and exposes enough identity to explain the decision. Non-BSC routes require fresh proof, connected health, matching route/network/asset digests, compatible provider, valid address/memo; legacy native BNB path may be ready without proof. Ref: `manual-monitoring-readiness.test.ts`: 140-157, 255-285.

MON-02 - Fail closed on stale or mismatched monitoring proof

Role/route: Owner; `POST /admin/blockchain-monitoring/networks/:id/test`, then `POST /admin/blockchain-monitoring/setup/enable-ready`. Preconditions: sandbox Polygon route initially healthy, then test fixture/controlled provider can return stale health or changed token contract. Steps:

1. GET setup routes and identify the Polygon route.
2. POST its `/test`; verify health/proof is captured.
3. Change only the configured contract, endpoint, provider compatibility, or wait beyond proof freshness in a sandbox.
4. POST `enable-ready`.

Expected: changed identity/secret/route digest, stale health, incompatible provider, invalid address/memo, disabled asset/network, and mismatched monitor route remain unavailable; enable-ready does not enable them. Ref: manual-monitoring-readiness.test.ts:49-109,140-152,287-292 .

MON-03 - TRON, native/token, and BSC edge validation

Role/route: Owner; POST /admin/blockchain-monitoring/networks/:id/test; POST /admin/blockchain-monitoring/setup/enable-selected. Preconditions: sandbox routes for TRC20, BNB, BEP20; no real transfer. Steps:

- [illegible]

Expected: valid TRON identity can become ready; invalid Base58Check/address or malformed identity stays disabled. Native and token identities are not interchangeable. BNB/BEP20 canonical EVM addresses validate; BEP20 token requires proof. Ref: `manual-monitoring-readiness.test.ts:294-321,356-381,384-412`.

MON-04 - Registration gaps and safe activation

Role/route: Operator; GET /admin/blockchain-monitoring/registration-gaps, POST /admin/blockchain-monitoring/registration-gaps/{orderId}/activate. Preconditions: sandbox order whose eligible route has no watch; use a non-financial fixture order. Steps:

1. GET registration gaps and select an order ID returned by the API.
2. POST activation once.
3. GET gaps and watches (`/admin/blockchain-monitoring/watches`).
4. POST activation a second time and inspect status.

Expected: first activation creates/activates exactly one watch; second is idempotent or reports already active, never duplicates. Missing prerequisites produce a clear 4xx/409 rather than an unsafe watch. Ref: watch-registration-integration.test.ts:128+ and blockchain-monitoring.ts:35-38,334-342.

MON-05 - Review a detected match without real funds

Role/route: Operator; GET /admin/blockchain-monitoring/matches, POST /admin/blockchain-monitoring/matches/{id}/review. Preconditions: sandbox adapter fixture emits a deterministic match; no chain transfer. Steps:

1. GET matches and filter to the fixture watch/order.
2. POST review with the documented review payload/status.
3. GET matches again and inspect audit/status fields.
4. Repeat review with an invalid transition or unknown ID.

Expected: valid review changes only the selected match and records reviewer/time; invalid transition or ID returns validation/404/409 and leaves state unchanged. Ref: [blockchain-monitoring.ts:344-354](#) , [openapi.yaml:1430-1451](#) .

QEX-06 - Quickex provider config, pairs, and diagnostics

Role/route: Owner/operator; GET /quickex/config, GET /quickex/pairs, GET /quickex/quote, GET /quickex/admin/diagnostics, POST /quickex/admin/credentials/test. Preconditions: sandbox Quickex credentials or deliberately absent credentials. Steps:

1. GET config and pairs; choose a supported pair from the returned list.
2. Request a quote using that pair and a small sandbox amount.

3. GET diagnostics and POST credential test without changing credentials.
4. Repeat with unsupported pair and missing credentials.

Expected: config reports provider/rate mode safely (no secret); supported pair returns quote; unsupported pair is a documented 4xx; absent/invalid credentials report a non-ready authentication/reachability state, not a false healthy result. Ref: `openapi.yaml:2977-3139`, `quickex.test.ts`, `quickex-convert-default-pair.test.ts`.

QEX-07 - Quickex order lifecycle and provider synchronization

Role/route: Customer/operator; `POST /quickex/create-order`, `GET /quickex/orders/{id}/status`, `POST /quickex/validate-address`. Preconditions: Quickex sandbox account/provider; no production payout. Steps:

1. Validate a sandbox destination address and an invalid address.
2. Create one sandbox order from a supported pair using the returned quote data.
3. Poll status until terminal or provider timeout.
4. Query status with another customer/session and an unknown ID.

Expected: valid address/order returns provider order ID and status; provider preflight can be unavailable even when the signed create route is available, and invalid address must be rejected by authoritative signed order creation; status transitions are normalized and ownership is enforced; unknown/foreign IDs return 404/403. Ref: `openapi.yaml:3026-3063`, `quickex.test.ts`, `customer-order-scope.test.ts`.

WBIT-08 - WhiteBIT credential and balance health

Role/route: Owner/operator; `GET /admin/whitebit/balance`, `GET /admin/whitebit/assets/preview`. Preconditions: sandbox WhiteBIT credentials, then a run with credentials unset; never use a funded production account. Steps:

1. GET balance and asset preview with valid sandbox credentials.
2. Inspect credential source, provider reachability, verification state, and catalog errors.
3. Repeat with credentials absent or provider unavailable.

Expected: valid credentials return balance/catalog; absent credentials produce documented 503/not-configured (not empty-success); provider outage produces 502 and does not alter local mappings. Ref: `openapi.yaml:1101-1128`, `whitebit.ts:1234-1246,1417-1427`, `whitebit-integration.test.ts`.

WBIT-09 - WhiteBIT catalog mapping preview/import protection

Role/route: Operator then owner; `GET /admin/whitebit/assets/preview`, `POST /admin/whitebit/assets/import`, `GET /admin/crypto-networks/whitebit-auto-mapping/preview`, `POST ../apply`. Preconditions: sandbox catalog includes one missing asset, one existing mapped asset, and one Customer Deposits-enabled route. Steps:

1. GET both previews and save the proposed identities.
2. Confirm preview does not switch provider, disable deposits, or replace saved addresses.
3. POST import/apply with only reviewed unique entries.
4. Re-submit stale/duplicate proof or attempt to map a protected route.

Expected: only reviewed valid identities are saved; protected existing mappings/routes remain unchanged; stale proof is rejected and unrelated route proofs survive. Ref: `openapi.yaml:2170-2200`, `whitebit-swap-integration.test.ts`, `customer-deposit-eligibility.test.ts`.

WBIT-10 - WhiteBIT reconciliation/webhook idempotency

Role/route: Owner/provider simulator; `POST /admin/whitebit/reconcile`, `POST /webhooks/whitebit`. Preconditions: signed synthetic webhook fixture and isolated WhiteBIT history fixture; no real deposit. Steps:

1. POST reconcile and inspect counts/results.

2. POST one signed webhook with required `x-txc-payload`, `x-txc-signature`, and `x-txc-apikey` headers.
3. Replay the identical webhook and send one with altered signature/key/content type.
4. GET relevant order/deposit status through the documented order endpoint.

Expected: first event is accepted and correlated once; replay is idempotent; bad signature/key/content type is 401/4xx and creates no ledger duplicate; unconfigured credentials yield 503. Ref: `whitebit.ts:1297,1496-1504`, `openapi.yaml:1087-1183`, `whitebit-history-worker.test.ts`, `whitebit-swap-integration.test.ts`.

MAIL-11 - Customer order status email opt-in/out

Role/route: Signed-in customer; `GET/PATCH /account/orders/{id}/notifications` (OpenAPI describes update; verify actual method accepted by server). Preconditions: owned sandbox order and verified customer email; notification delivery sink/mailbox, not a real customer. Steps:

1. GET owned order and current notification preference.
2. Enable status-change notifications using the documented body.
3. Trigger a sandbox status transition through the operator test workflow, then inspect sink/mailbox.
4. Disable notifications and trigger another transition; repeat with unverified email and foreign order.

Expected: one deduplicated email per eligible transition while enabled; none after opt-out; unverified email is rejected with the documented error; foreign order is forbidden/not found. Ref: `openapi.yaml:1043-1085`, `customer-status-notifications.test.ts`, `notification-policy.ts`.

MAIL-12 - Admin email settings, templates, and safe test delivery

Role/route: Owner; `GET/PUT /admin/notification-settings`, `GET/PUT /admin/notification-settings/email-templates`, `POST /admin/notification-settings/test-email`, `POST /admin/notification-settings/email-templates/test`. Preconditions: staging mail sink and owner auth. Steps:

1. GET settings/templates and record current version.
2. PUT a harmless staging-only template edit; GET again.
3. POST test email to an approved sink address.
4. Submit invalid template/recipient and retry as operator-only auth.

Expected: settings/template update is reflected and versioned; test email is observable in sink with correct subject/template; invalid data is 4xx; non-owner cannot read/write/test. Ref: `notification-settings.ts:95-147,255-288`, `customer-status-notifications.test.ts`.

REF-13 - Referral attribution and customer/admin settlement views

Role/route: Customer/operator; `GET /affiliate/referral/{code}`, `/account/affiliate`, `/account/affiliate/commissions`, `/account/affiliate/referrals`, `/account/affiliate/payouts`, `GET /admin/affiliate/overview`, `/admin/affiliate/accounts`, `/admin/affiliate/commissions`. Preconditions: isolated affiliate fixtures with two customers and deterministic completed/expired orders. Steps:

1. Resolve a valid and invalid referral code.
2. Bind/inspect attribution as the sandbox customer.
3. GET commission/referral/payout pages with `page=1&pageSize=2`, then page 2.
4. Compare admin overview/accounts/commissions and attempt cross-customer access.

Expected: valid code resolves without leaking private identity; pagination is stable; only eligible completed activity creates commission, expired/refunded activity does not; customer sees only own data and admin sees aggregate/detail data. Ref: `openapi.yaml:3139-3303`, `affiliate.test.ts:220-297,409-412`.

REF-14 - Controlled referral payout settlement state machine

Role/route: Operator; GET /admin/affiliate/payouts, PATCH /admin/affiliate/payouts/{id}. Preconditions: isolated payout request in requested state; settlement is simulation only. Steps:

1. PATCH requested -> approved.
2. PATCH approved -> processing.
3. PATCH processing -> paid with synthetic sandbox TXID such as sandbox-tx-001.
4. Try paid without TXID, repeat a transition, and try an out-of-order status.

Expected: each legal transition returns updated status and audit record; paid requires nonblank TXID; duplicate/out-of-order transitions return 409 PAYOUT_STATE_CONFLICT; no chain transfer is initiated by this test. Ref: affiliate.ts:85, openapi.yaml:3226-3264.

CFG-15 - Workspace config preview, apply, and production safety

Role/route: Owner; POST /admin/workspace-config/preview, POST /admin/workspace-config/apply. Preconditions: export from development/staging via src/scripts/export-workspace-config.ts; production backup and maintenance approval; do not apply during this test. Steps:

1. POST preview with the export and inspect changed assets/networks/payment links/pages and warnings.
2. Confirm IDs are projected by asset/code and network identity, not blindly copied.
3. Preview malformed duplicate object path, invalid SHA-256, missing reference, and string manualWalletTrackingEnabled.
4. In staging only, apply reviewed snapshot, then GET public config and admin setup routes.

Expected: preview is non-mutating and gives deterministic diff; malformed bundles/references are rejected; apply is atomic, preserves explicit wallet tracking OFF, safely reactivates a payment method only when a reviewed enabled link requires it, and production has an explicit diff/approval gate. Ref: workspace-config-sync.ts:322-387, workspace-config-sync.test.ts:33-37,39-94,179-241,243-259, exports/quickxchange-production-config-dry-run.json.

OPS-16 - Health/readiness and runtime/deployment smoke test

Role/route: Deployment operator; GET /healthz, GET /readyz. Preconditions: deployed API process, staging DB/migrations and secrets configured. Steps:

1. Call /healthz without auth and record HTTP status/body.
2. Call /readyz with DB/provider configuration healthy.
3. In an isolated deployment, remove/disable a required DB connection or migration and call again.
4. Start using the documented production start command from scripts/production-api-start.test.mjs; inspect process logs/exit code only.

Expected: health reports process liveness; readiness runs a DB select and returns 503 when the DB is unavailable; startup does not silently use development configuration or skip migrations. Ref: artifacts/api-server/src/routes/health.ts, lib/api-spec/openapi.yaml, scripts/production-api-start.test.mjs, lib/db/production-monitoring-migrations.test.mjs.

OPS-17 - Admin orders XML export and public sitemap

Role: Admin with orders.export, and SEO/deployment operator; GET /admin/orders.xml (API) plus GET /sitemap.xml (web static asset). Preconditions: deployed web artifact; no API mutation. Steps:

1. As authorized Admin, click Export XML on /admin/orders and parse the downloaded orders XML; as a guest, request /api/admin/orders.xml and confirm access is denied. 2. GET /sitemap.xml from the web artifact deployment URL.

2. Validate sitemap XML, content type and whether absolute URLs match the deployed environment; report stale domain entries as a defect.
3. Compare visible public content/navigation/blog pages with sitemap entries.
4. Probe undocumented public /feed.xml and /rss.xml paths only to confirm no feed is promised.

Expected: authorized XML export downloads an orders document and guest cannot access it; static sitemap is reachable and valid, and stale domain entries are a defect; undocumented public feeds need not exist. API never exposes admin/customer data through unauthenticated XML. Ref: widget public/sitemap.xml , openapi.yaml:3327-3428 ; Admin export: artifacts/api-server/src/routes/exchange.ts , lib/api-spec/openapi.yaml ; static sitemap: widget public/sitemap.xml .

DEP-18 - Account-only Deposits decommission/compatibility guard

Role/owner/operator; public GET /account/orders , /account/orders/{id} , and deposit creation paths in exchange.ts . Preconditions: staging account with legacy deposit-shaped order and a current order; no DB edits. Steps:

1. Use API/UI account order history and search for a standalone “Deposits” navigation/page or endpoint.
2. GET both order records with the owning account and a different account.
3. Attempt a new normal order requiring a customer crypto deposit, with route disabled/not-ready, then with a ready provider route.
4. Check OpenAPI and server logs for a separate account-only Deposits contract.

Expected: no standalone Deposits feature is exposed; legacy records remain readable only through scoped order compatibility; foreign records are hidden/403; disabled/unready routes return 422 DESK_CRYPT0_DEPOSIT_UNAVAILABLE ; ready routes expose current deposit instructions. Do not delete records or mutate DB. Ref: exchange.ts:1190,1346,1423-1433 , order-history.ts:229-272 , openapi.yaml:955-1043 , customer-deposit-eligibility.test.ts ; tracked follow-up

END

Recording and sign-off

For each case above record: ID, environment/build, role, test data, observed result, expected result, PASS/FAIL/BLOCKED, redacted evidence and defect reference. A BLOCKED case is not a pass; identify the missing permission, provider fixture, test-only message sink, configured route or approval. Rerun affected cases after a fix and check neighboring role and route boundaries.

Release recommendation: require explicit sign-off for customer funds, authentication/authorization, order idempotency, tracking capabilities, provider statuses, and legacy deposit safety. This guide does not authorize destructive production data changes or publishing.